

Integer Representation

CSC 230 : C and Software Tools

NC State Department of Computer
Science

Topics for Today

- Unsigned Numbers
 - Arithmetic
 - Overflow
- Two's Complement
 - Arithmetic and Overflow
- Hexadecimal and Octal

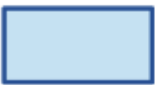
Integer Type Representation

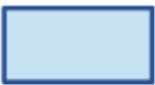
- The standard makes few guarantees about how these types should be represented.
- It does for **unsigned types**.

long double 

double 

float 

signed long long 

signed long 

signed int 

signed short 

signed char 

_Bool 

void •

 unsigned long long

 unsigned long

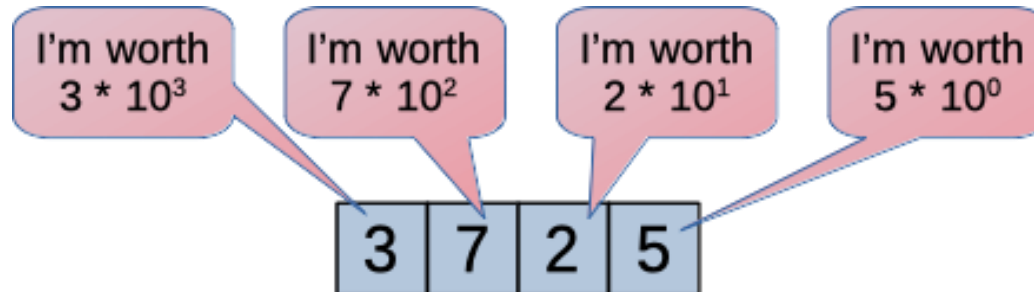
 unsigned int

 unsigned short

 unsigned char

Unsigned Values in Binary

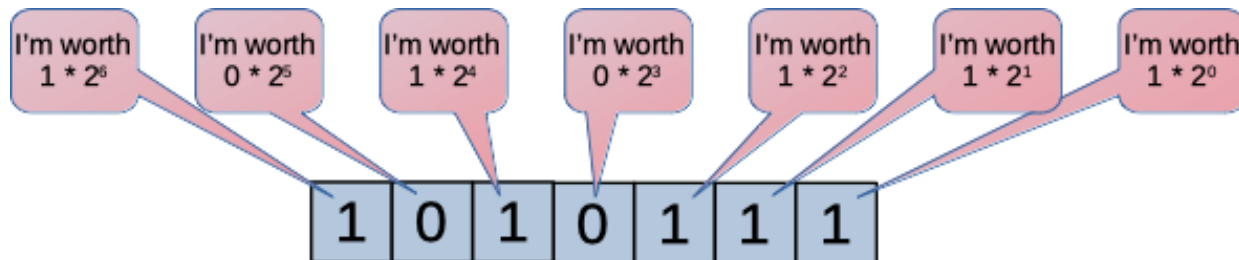
- For humans (in base ten)
 - Each digit can have a value of 0 .. 9
 - Each digit is worth a power of 10.
- Consider a (base ten) number like 3725



- This is called a ***positional number system***

Unsigned Values in Binary

- Computers are really good at storing on / off
- Binary numbers use this store arbitrary values
 - We just have 2 digits, 0 (i.e., off) and 1 (on)
 - Place value uses powers of 2 rather than powers of ten.
- Consider a binary number like 1010111



Counting in Binary

[illegible]

Counting in Binary

0	0	0	0	0	0	0	0	$= 0_{10}$
0	0	0	0	0	0	0	1	$= 1_{10}$

Counting in Binary

0	0	0	0	0	0	0	0	0	= 0 ₁₀
0	0	0	0	0	0	0	0	1	= 1 ₁₀
0	0	0	0	0	0	0	1	0	= 2 ₁₀

Counting in Binary

0	0	0	0	0	0	0	0	0	= 0 ₁₀
0	0	0	0	0	0	0	0	1	= 1 ₁₀
0	0	0	0	0	0	1	0		= 2 ₁₀
0	0	0	0	0	0	1	1		= 3 ₁₀

Counting in Binary

0	0	0	0	0	0	0	0		$= 0_{10}$
0	0	0	0	0	0	0	0	1	$= 1_{10}$
0	0	0	0	0	0	0	1	0	$= 2_{10}$
0	0	0	0	0	0	0	1	1	$= 3_{10}$
0	0	0	0	0	1	0	0		$= 4_{10}$

Counting in Binary

0	0	0	0	0	0	0	0		$= 0_{10}$
0	0	0	0	0	0	0	0	1	$= 1_{10}$
0	0	0	0	0	0	0	1	0	$= 2_{10}$
0	0	0	0	0	0	0	1	1	$= 3_{10}$
0	0	0	0	0	0	1	0	0	$= 4_{10}$
0	0	0	0	0	0	1	0	1	$= 5_{10}$

Counting in Binary

0	0	0	0	0	0	0	0		$= 0_{10}$
0	0	0	0	0	0	0	0	1	$= 1_{10}$
0	0	0	0	0	0	0	1	0	$= 2_{10}$
0	0	0	0	0	0	0	1	1	$= 3_{10}$
0	0	0	0	0	0	1	0	0	$= 4_{10}$
0	0	0	0	0	0	1	0	1	$= 5_{10}$
0	0	0	0	0	0	1	1	0	$= 6_{10}$

Counting in Binary

0	0	0	0	0	0	0	0		$= 0_{10}$
0	0	0	0	0	0	0	0	1	$= 1_{10}$
0	0	0	0	0	0	0	1	0	$= 2_{10}$
0	0	0	0	0	0	0	1	1	$= 3_{10}$
0	0	0	0	0	0	1	0	0	$= 4_{10}$
0	0	0	0	0	0	1	0	1	$= 5_{10}$
0	0	0	0	0	0	1	1	0	$= 6_{10}$
0	0	0	0	0	0	1	1	1	$= 7_{10}$

Counting in Binary

0	0	0	0	0	0	0	0		$= 0_{10}$
0	0	0	0	0	0	0	1		$= 1_{10}$
0	0	0	0	0	0	1	0		$= 2_{10}$
0	0	0	0	0	0	1	1		$= 3_{10}$
0	0	0	0	0	1	0	0		$= 4_{10}$
0	0	0	0	0	1	0	1		$= 5_{10}$
0	0	0	0	0	1	1	0		$= 6_{10}$
0	0	0	0	0	1	1	1		$= 7_{10}$
									
flip all the 1s on the right.									
0	0	0							

Counting in Binary

0	0	0	0	0	0	0	0	= 0 ₁₀
0	0	0	0	0	0	0	1	= 1 ₁₀
0	0	0	0	0	0	1	0	= 2 ₁₀
0	0	0	0	0	0	1	1	= 3 ₁₀
0	0	0	0	0	1	0	0	= 4 ₁₀
0	0	0	0	0	1	0	1	= 5 ₁₀
0	0	0	0	0	1	1	0	= 6 ₁₀
0	0	0	0	0	1	1	1	= 7 ₁₀

flip the first 0 to a 1

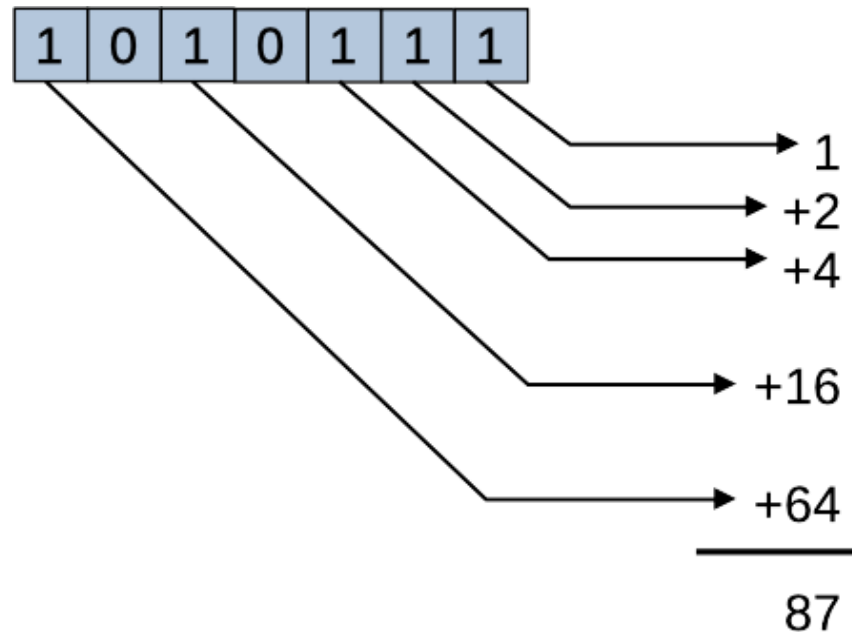
1	0	0	0
---	---	---	---

Counting in Binary

0	0	0	0	0	0	0	0	
								$= 0_{10}$
0	0	0	0	0	0	0	1	
								$= 1_{10}$
0	0	0	0	0	0	1	0	
								$= 2_{10}$
0	0	0	0	0	0	1	1	
								$= 3_{10}$
0	0	0	0	0	1	0	0	
								$= 4_{10}$
0	0	0	0	0	1	0	1	
								$= 5_{10}$
0	0	0	0	0	1	1	0	
								$= 6_{10}$
0	0	0	0	0	1	1	1	
								$= 7_{10}$
								
the rest stays the same								
0	0	0	0	1	0	0	0	
								$= 8_{10}$

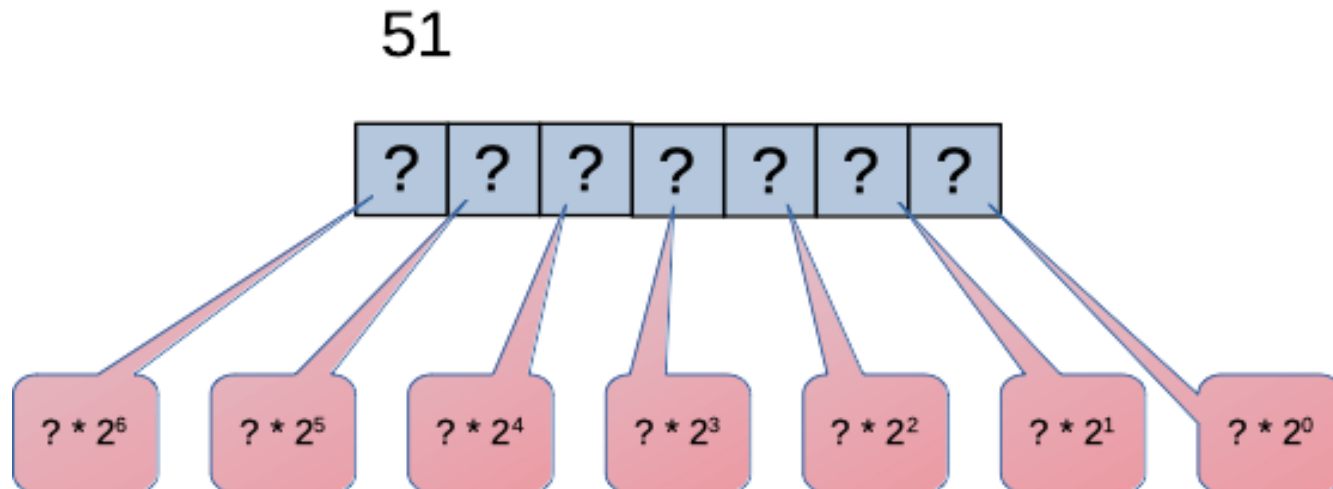
Binary to Decimal

- This is how **unsigned** values are stored in C
 - To see what a binary value is (in base ten)
 - ... we can just add up the value of all the one bits.



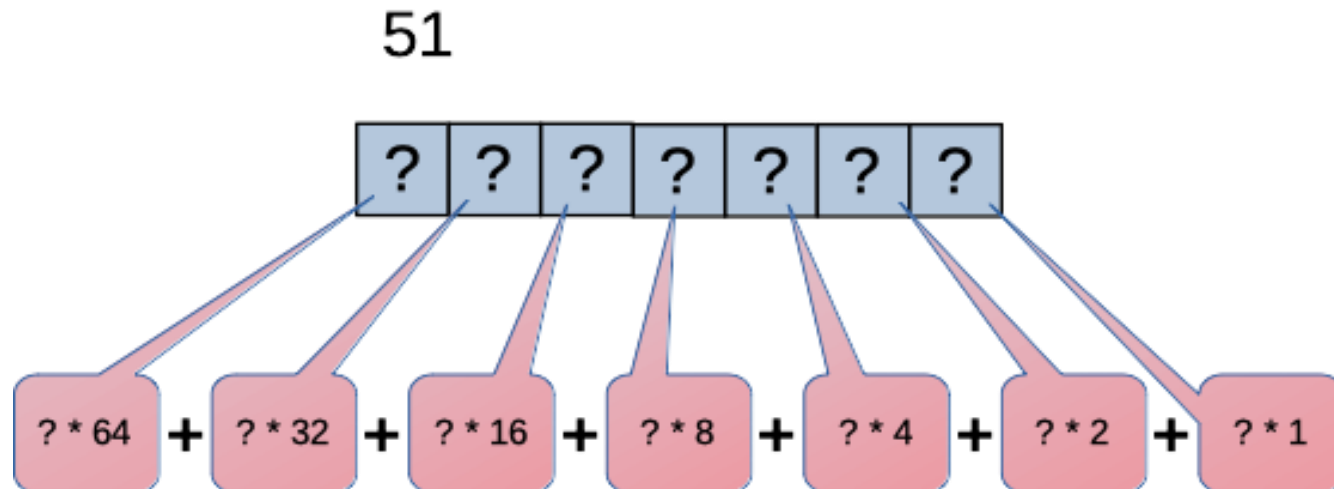
Decimal to Binary

- Given a number in base 10
- How can we figure out its value in binary?
- We'll see two common ways to do this.



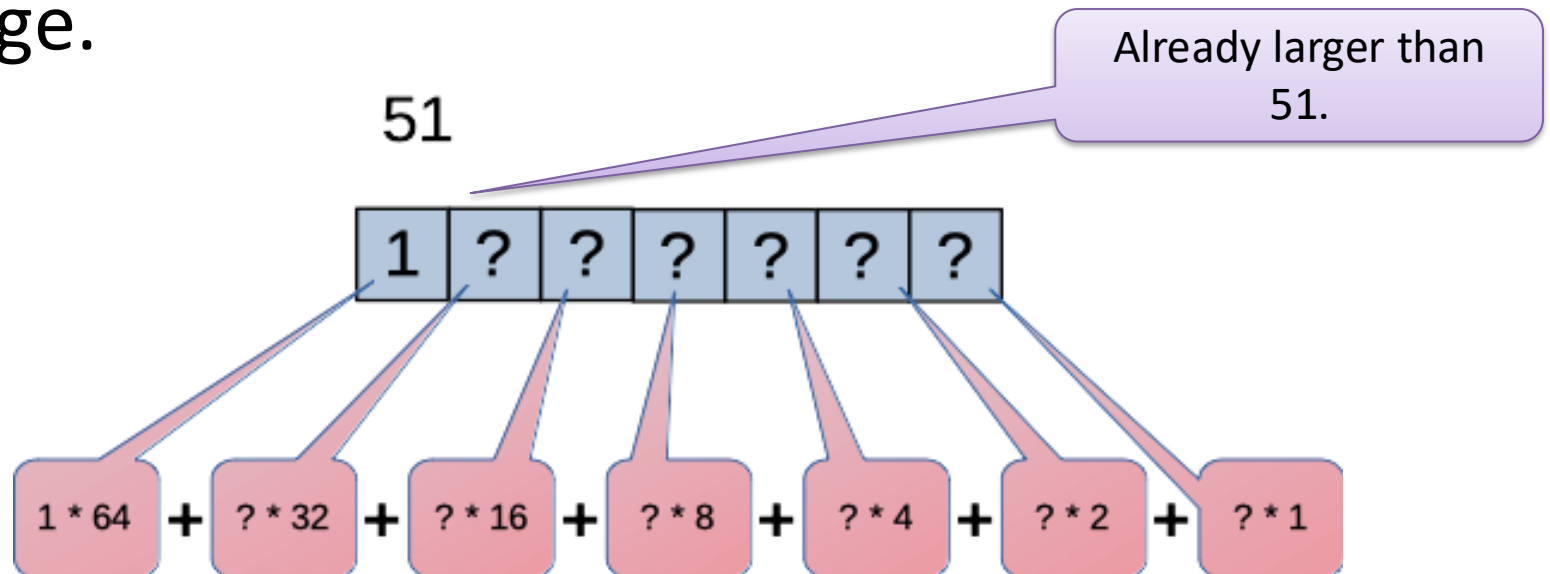
Decimal to Binary

- Each bit contributes to a sum.
 - Each one can be either 0 or 1.
 - If it's zero, it contributes nothing to the sum.
 - If it's one, it contributes a power of 2.



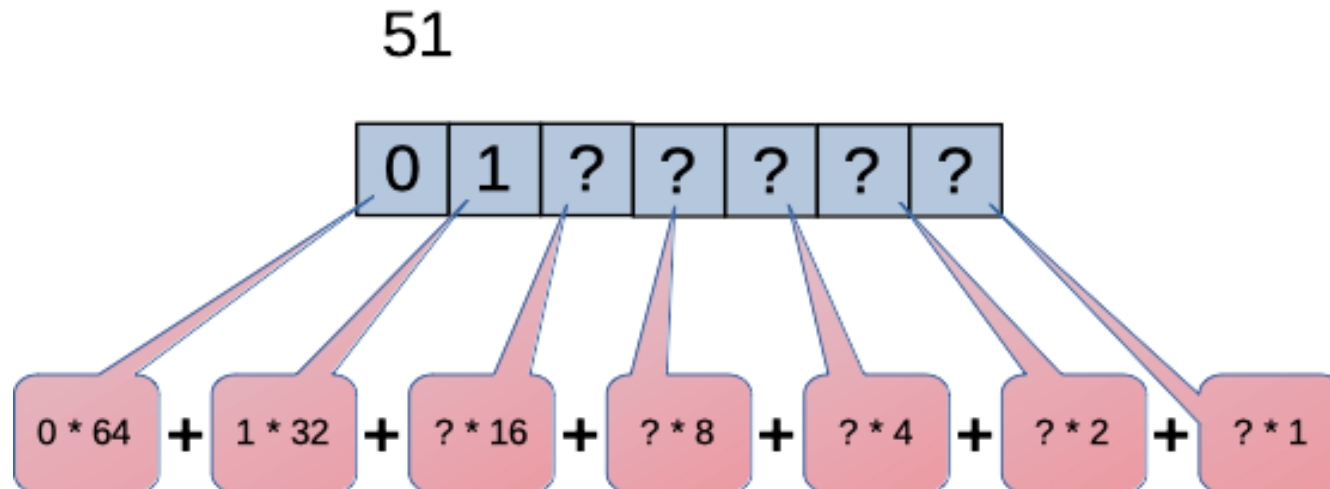
Decimal to Binary

- Try a 1 for the **most significant bit**.
 - The bit on the left.
- Here, if the 2^6 bit was 1, the sum would be too large.



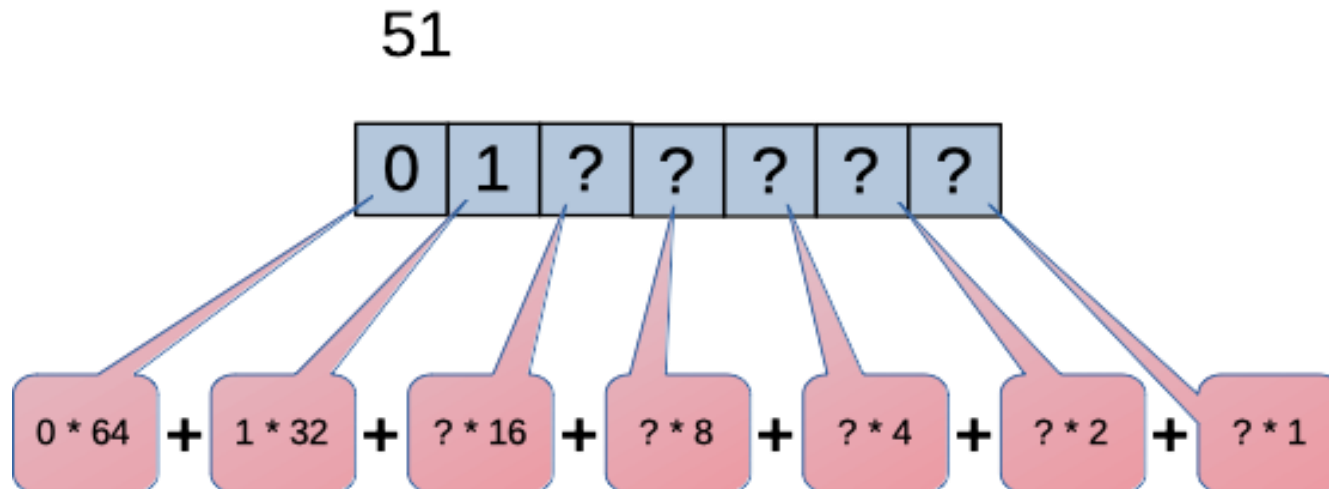
Decimal to Binary

- Try out a 1 for the 2^5 bit.



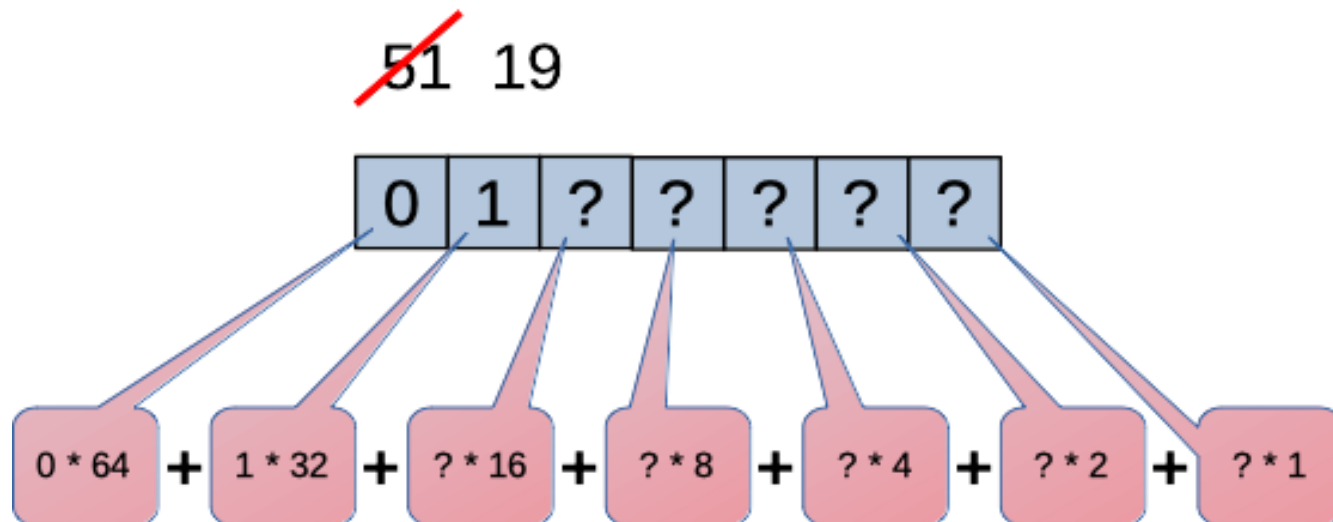
Decimal to Binary

- Try out a 1 for the 2^5 bit.
- That adds 32 to the sum.
 - That's less than (or equal to) 51.
 - ... and, all the remaining bits only add up to 31.
 - ... so the 2^5 bit **must** be a 1.



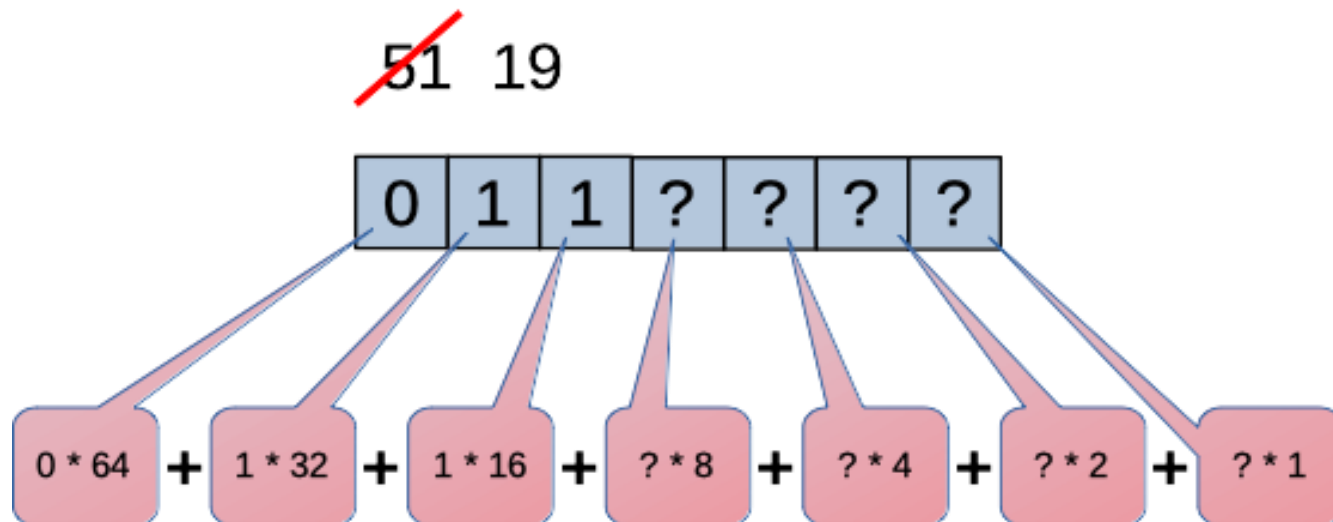
Decimal to Binary

- That accounts for 32 of the 51 we started with
 - Leaving 19 (of the original 51)
 - ... the remaining bits must add up to 19



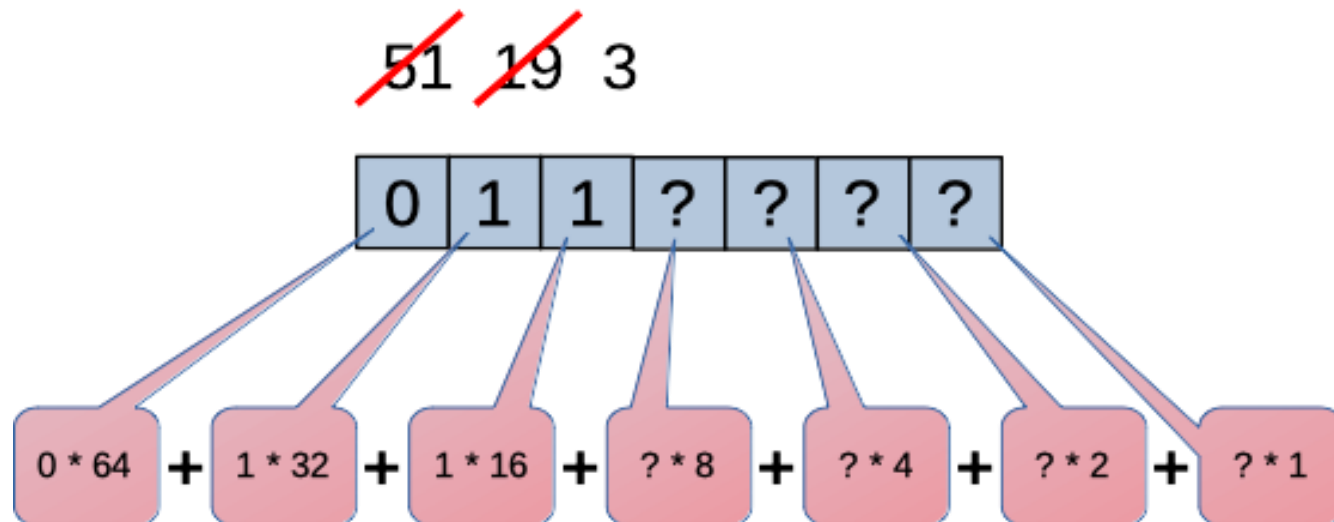
Decimal to Binary

- The 2^4 bit is worth 16
 - That doesn't exceed the 19 we have remaining.
 - ... and all remaining bits add up to just 15.
 - ... so the 2^4 bit must be a 1.



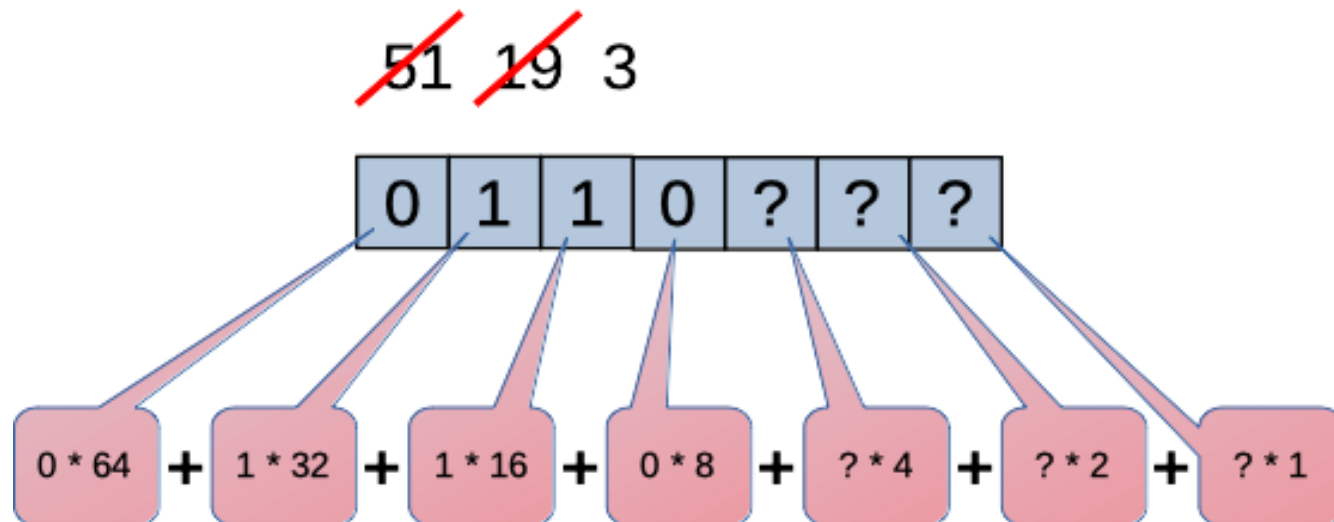
Decimal to Binary

- That leaves 3 for the rest of the bits.



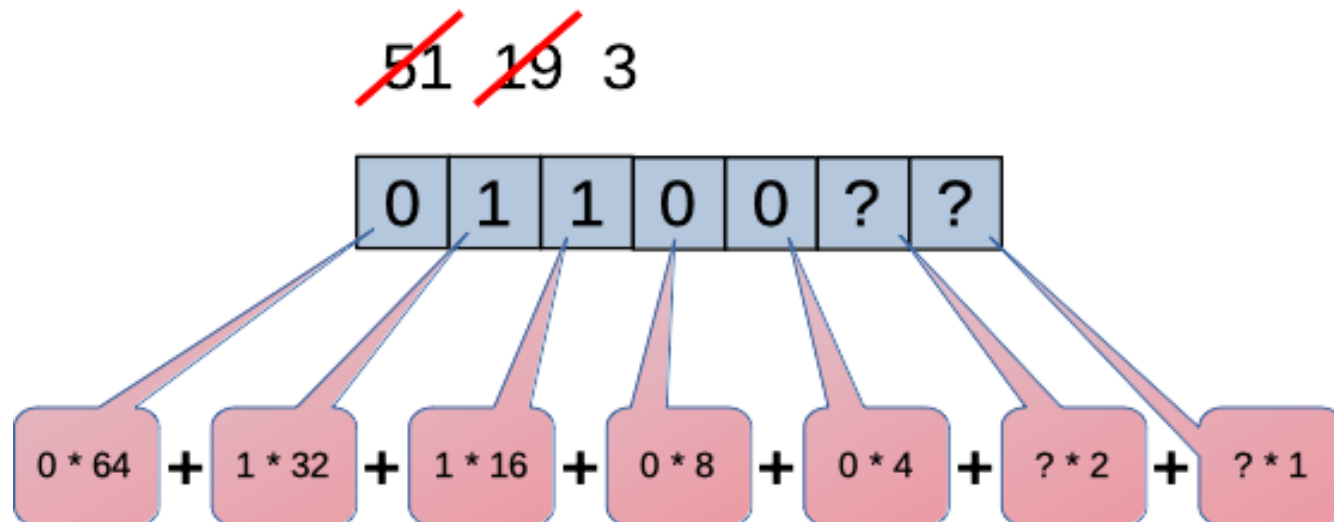
Decimal to Binary

- The 2^3 bit must be zero
 - Otherwise the sum would be too large.



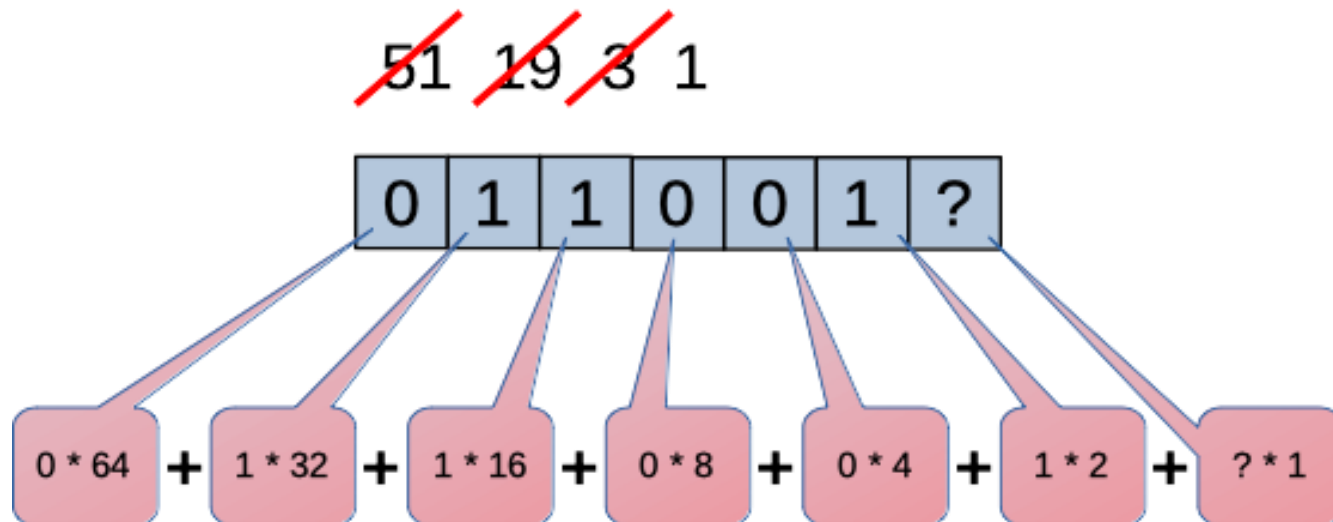
Decimal to Binary

- Likewise, the 2^2 bit must be zero



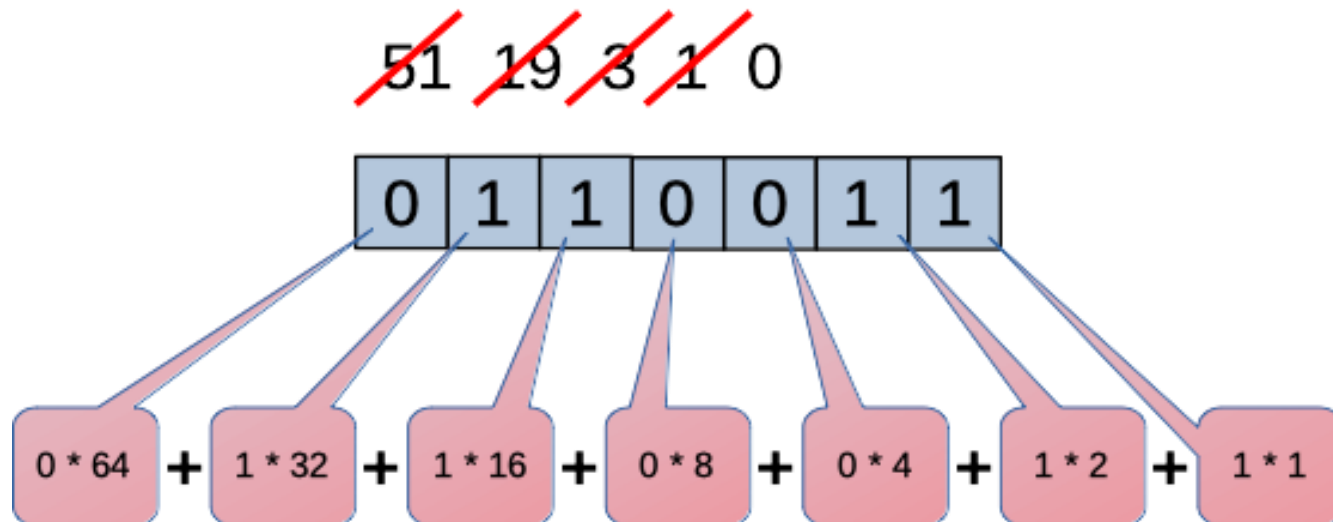
Decimal to Binary

- The 2^1 bit must be a 1.
 - It's worth $2^1 = 2$
- That leaves just 1 for the last bit.



Decimal to Binary

- The **least significant bit** must be a 1.
 - The one on the right.
- The value of all the bits add up to 51.



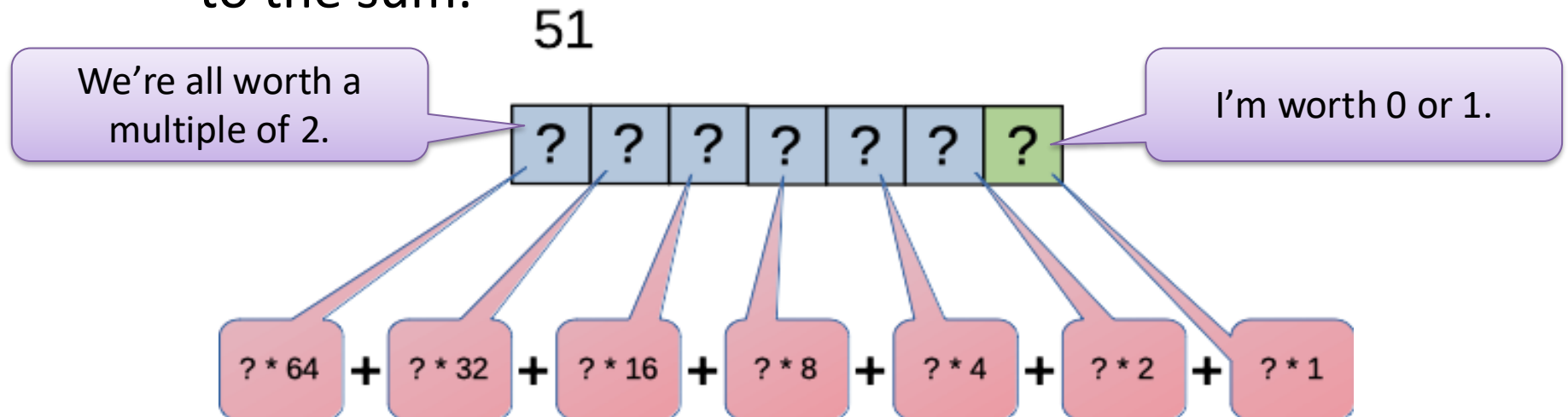
Decimal to Binary

- This technique is called **reduction of powers**
- How do you know what bit to start with?
 - Two answers
 - You can try out larger and larger powers of two
 - Until you find one that's larger than the value you're converting.
 - But, usually, you're working with a fixed number of bits, based on the type.
 - E.g., an 8-bit char, a 16-bit short or a 32-bit int.

Decimal to Binary

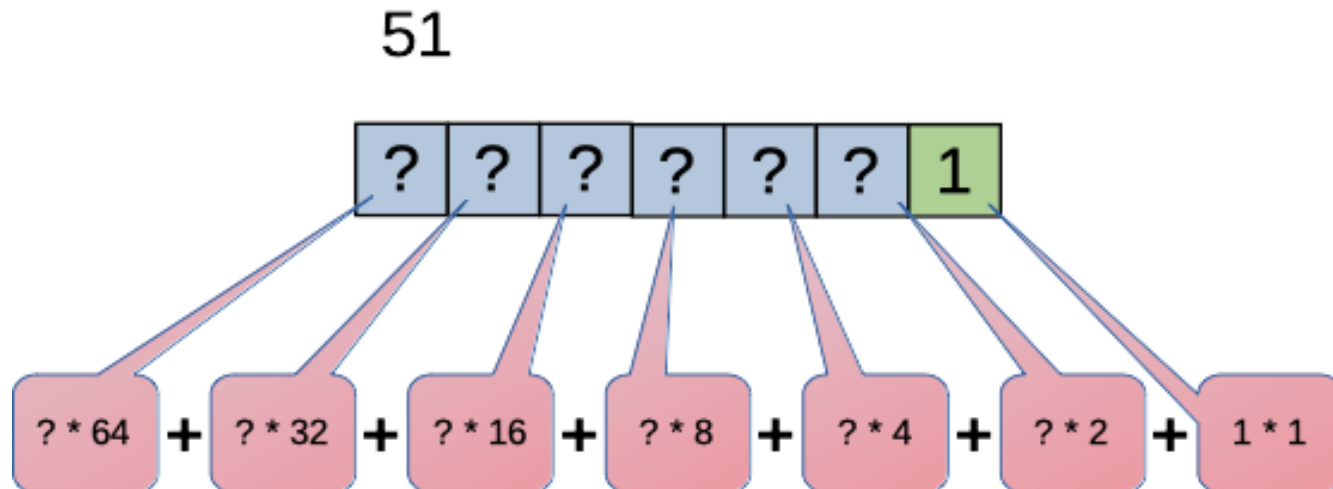
(a different technique)

- We can also convert to binary starting with the least significant bit.
 - If our value is odd, the least significant bit must be a 1
 - Why? Every other bit contributes some multiple of 2 to the sum.



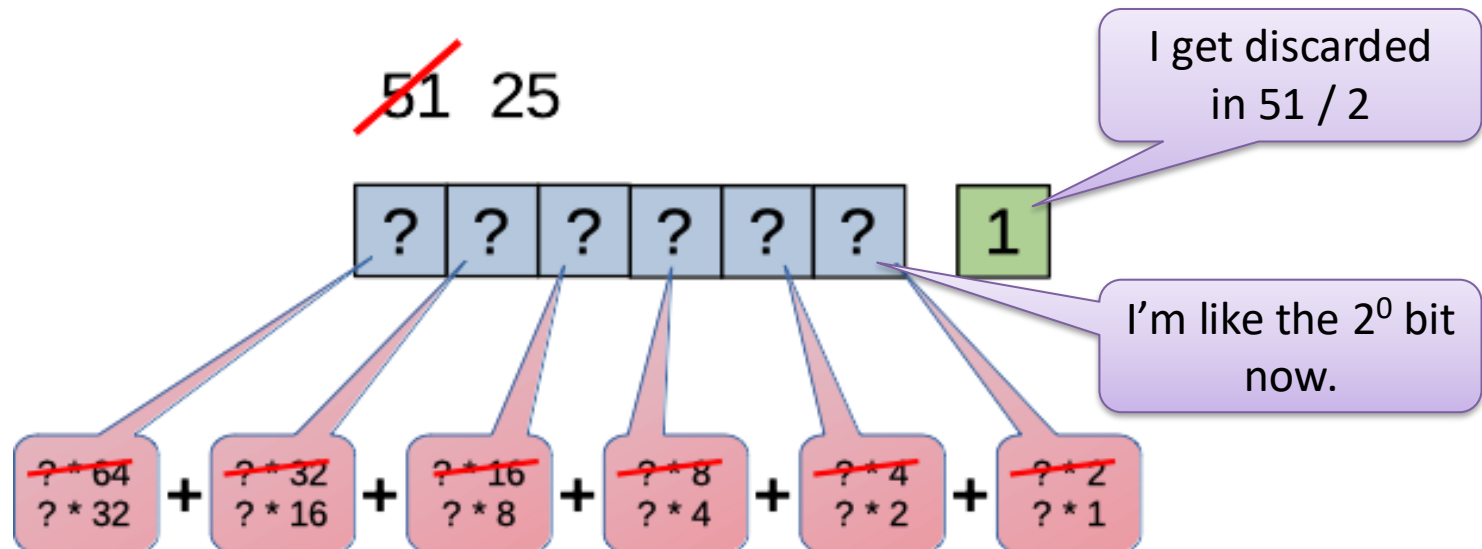
Decimal to Binary

- 51 is odd
 - So the least significant bit must be 1.
 - ... now, how do we figure out the rest of the bits?



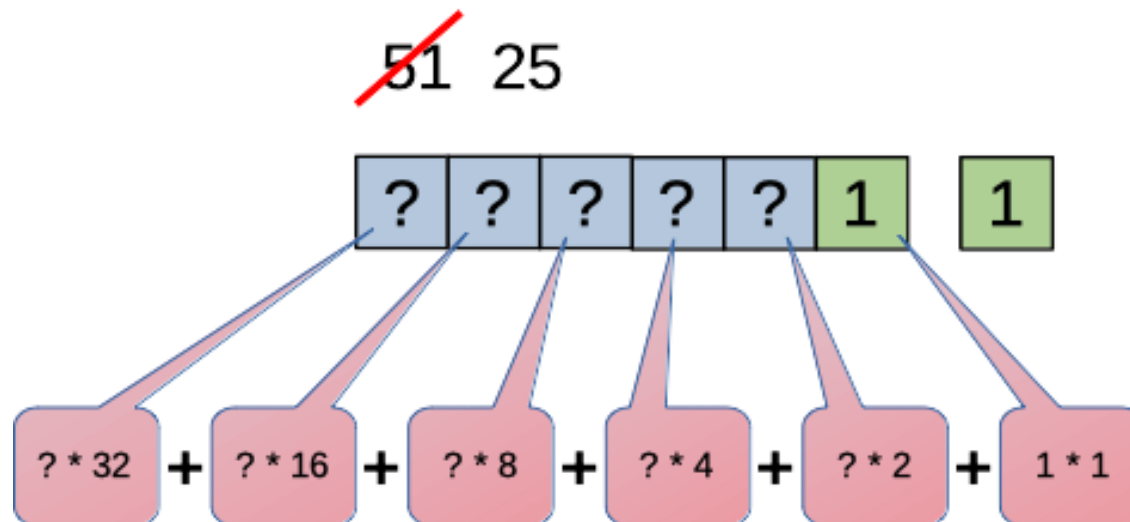
Decimal to Binary

- If we div by 2 (with truncation), that will
 - Discard the low-order bit ... it's just part of the remainder.
 - And, it will effectively slide all the other bits to the right.



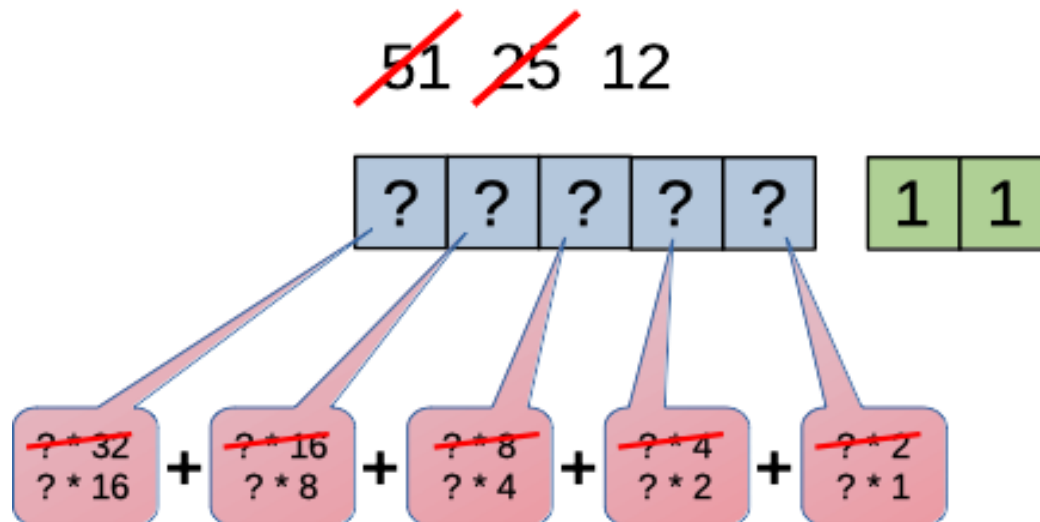
Decimal to Binary

- Now, we just need to convert 25 to binary.
- 25 is odd
 - ... so the least significant bit must be 1.



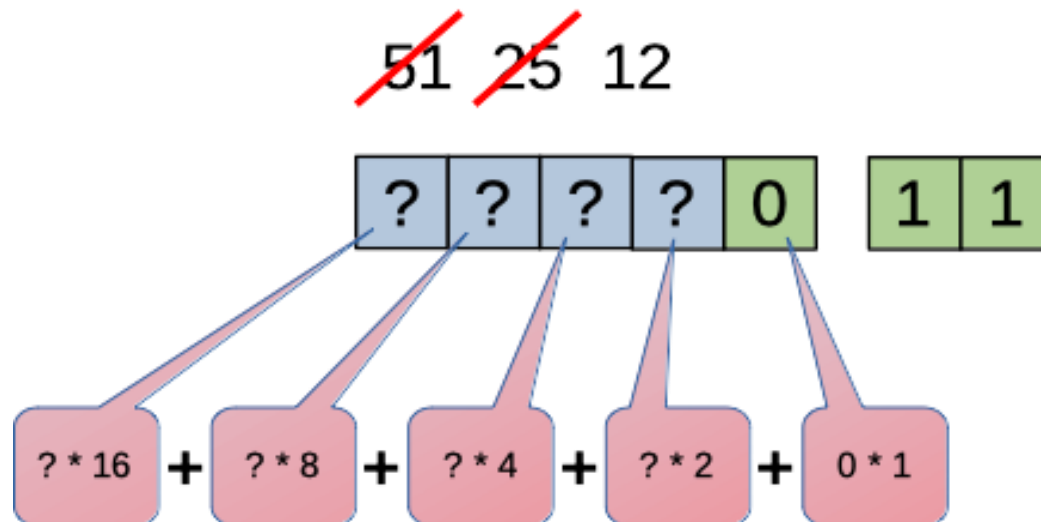
Decimal to Binary

- Dividing by 2 again
 - Effectively discards the least-significant bit
 - ... and slides all the other bits to the right.



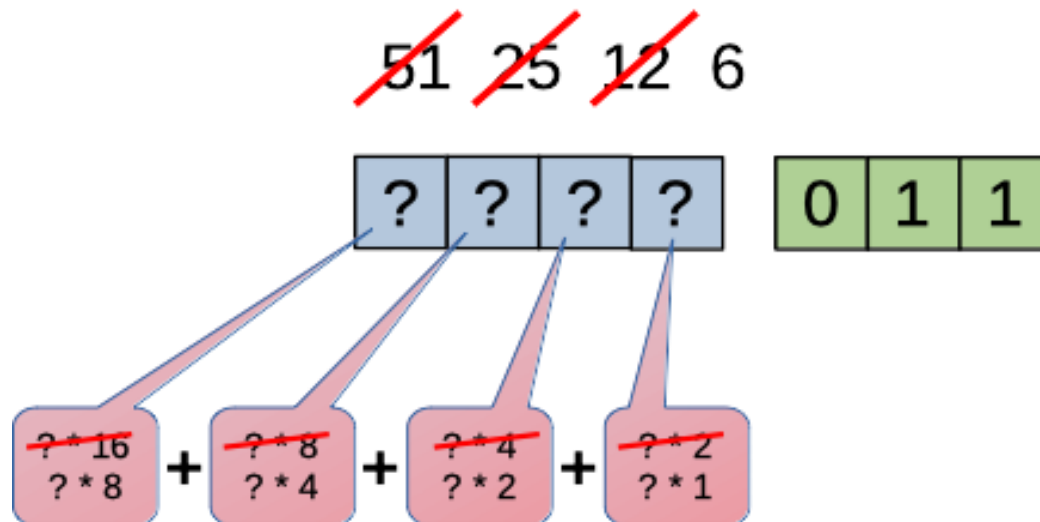
Decimal to Binary

- 12 is even ... so the next bit must be zero.



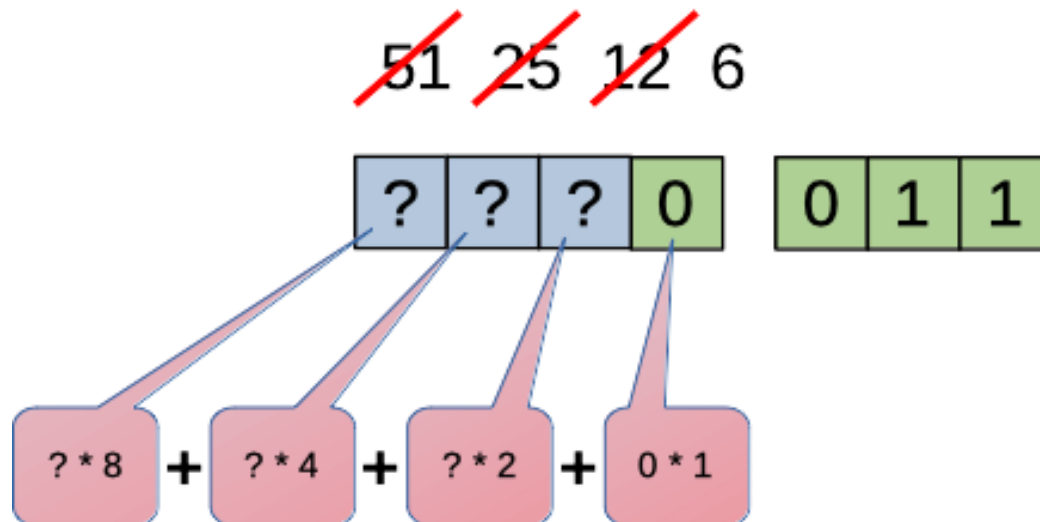
Decimal to Binary

- Dividing by 2 again gives us 6



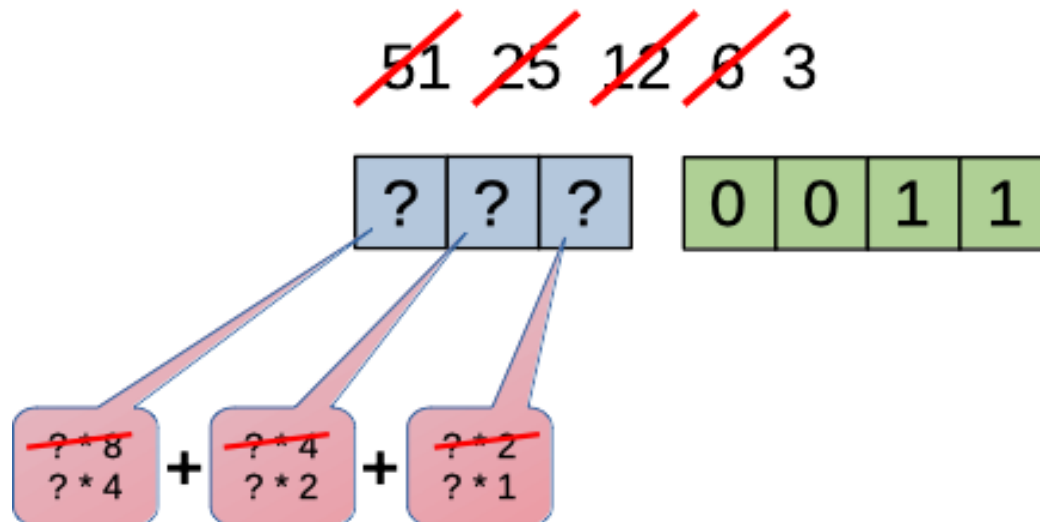
Decimal to Binary

- 6 is even, so the next bit must be zero.



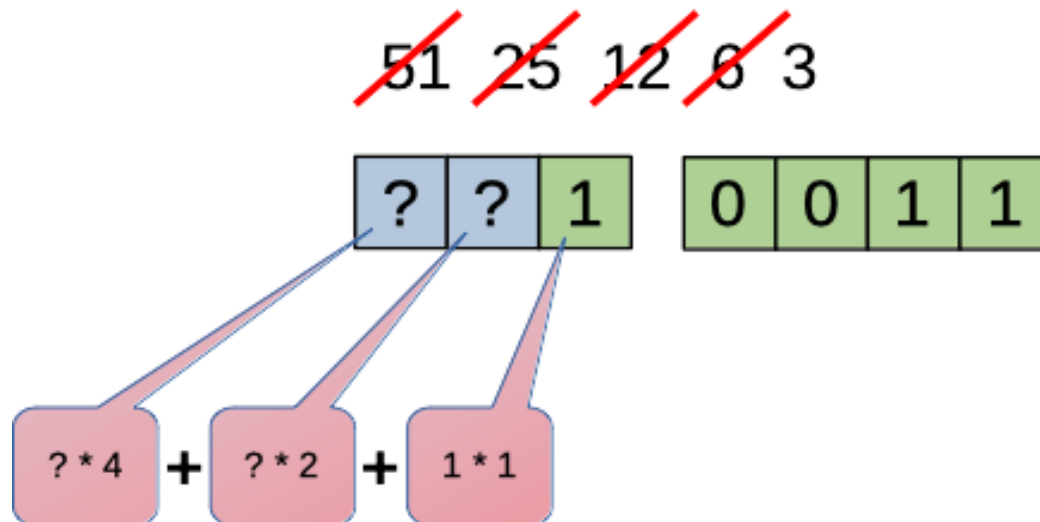
Decimal to Binary

- Dividing by 2 gives us 3.



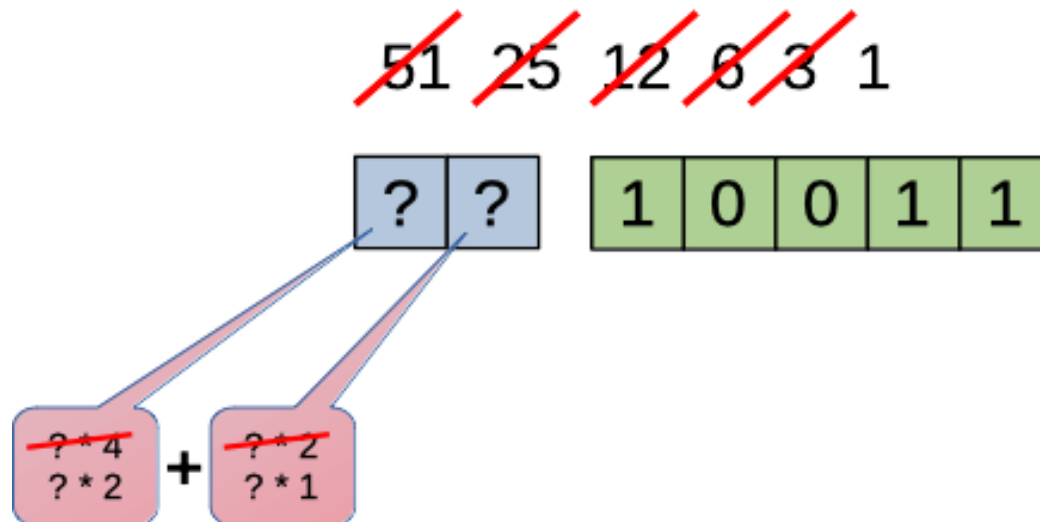
Decimal to Binary

- So, the next bit must be a one.



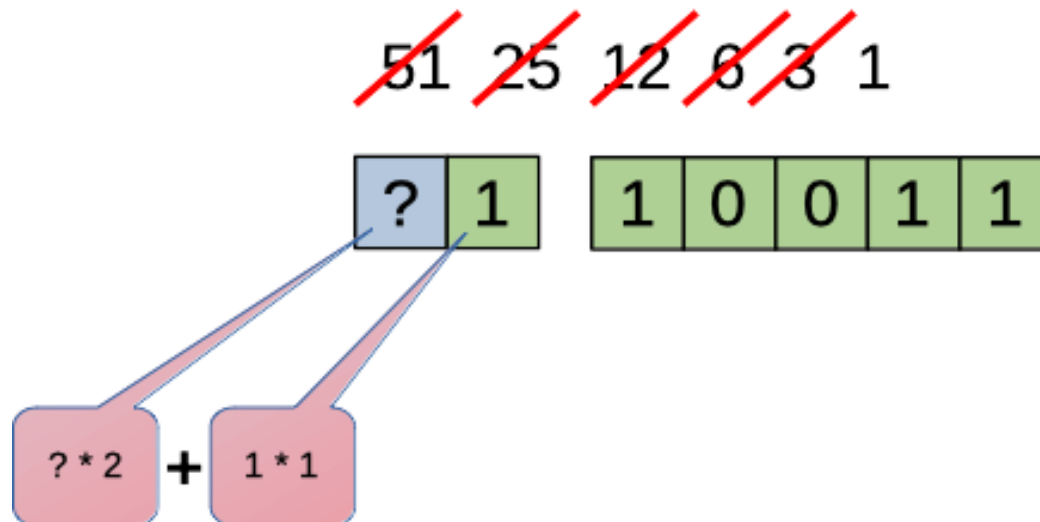
Decimal to Binary

- Dividing by 2 leaves 1.



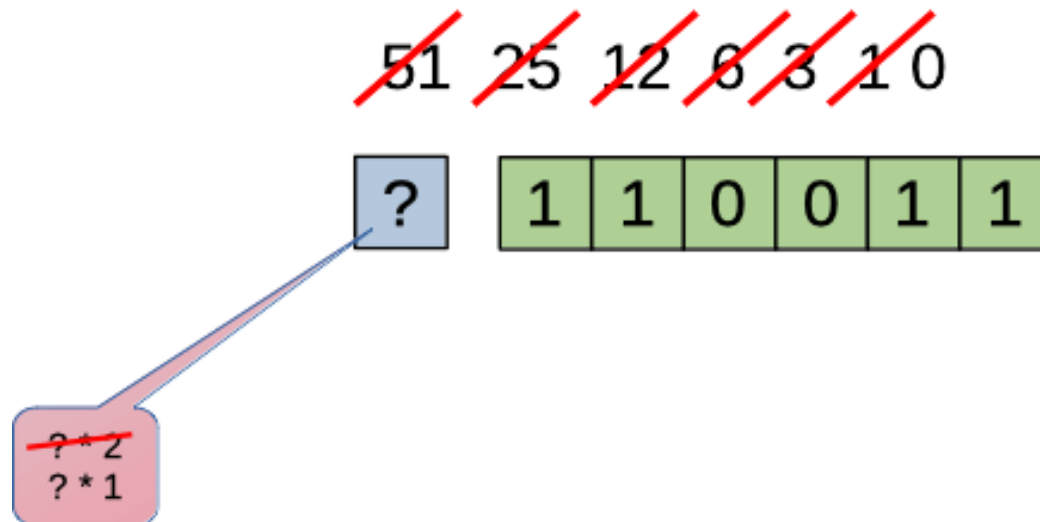
Decimal to Binary

- So, the next bit must be a one.



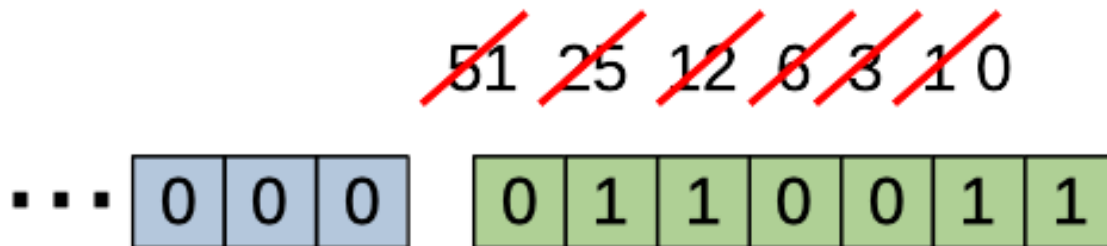
Decimal to Binary

- Now, dividing by 2 leaves zero.



Decimal to Binary

- The next bit must be a zero
- You could keep going, but all remaining bits would be zero.



Math in Binary

- Adding unsigned values is easy in binary.
 - Easier than in base ten
 - ... there are just two digit values.
- We can add bit-by-bit, starting from the low-order bit.

$$\begin{array}{r} 01010 \\ + 01100 \\ \hline \end{array}$$

Addition in Binary

- Adding unsigned values is easy in binary.
 - Easier than in base ten
 - ... there are just two digit values.
- We can add bit-by-bit, starting from the low-order bit.

$$\begin{array}{r} 01010 \\ + 01100 \\ \hline 0 \end{array}$$

Zero plus Zero is
Zero.

Addition in Binary

- Adding unsigned values is easy in binary.
 - Easier than in base ten
 - ... there are just two digit values.
- We can add bit-by-bit, starting from the low-order bit.

$$\begin{array}{r} 01010 \\ + 01100 \\ \hline 10 \end{array}$$

One plus Zero is
One

Addition in Binary

- Adding unsigned values is easy in binary.
 - Easier than in base ten
 - ... there are just two digit values.
- We can add bit-by-bit, starting from the low-order bit.

$$\begin{array}{r} 01010 \\ + 01100 \\ \hline 110 \end{array}$$

Addition in Binary

- Adding unsigned values is easy in binary.
 - Easier than in base ten
 - ... there are just two digit values.
- We can add bit-by-bit, starting from the low-order bit.

$$\begin{array}{r} 01010 \\ + 01100 \\ \hline 2110 \end{array}$$

2?

There's no 2 in binary.

Addition in Binary

- Adding unsigned values is easy in binary.
 - Easier than in base ten
 - ... there are just two digit values.
- We can add bit-by-bit, starting from the low-order bit.

$$\begin{array}{r} 1 \\ 01010 \\ + 01100 \\ \hline 0110 \end{array}$$

We have to carry into the next bit position.

Addition in Binary

- Adding unsigned values is easy in binary.
 - Easier than in base ten
 - ... there are just two digit values.
- We can add bit-by-bit, starting from the low-order bit.

$$\begin{array}{r} 1 \\ 01010 \\ + 01100 \\ \hline 10110 \end{array}$$

0 plus 0 plus the carry equals 1.

Rules for Binary Addition

- Add up bits, working from least significant to most significant.
- Propagate carry into subsequent bits.
- There are only 8 possible cases:

Carry from
the previous
column

$$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 0 \\ + 1 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1 \\ + 1 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1 \\ 0 \\ + 0 \\ \hline 1 \end{array}$$

$$\begin{array}{r} 1\ 1 \\ 0 \\ + 1 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1\ 1 \\ 1 \\ + 0 \\ \hline 0 \end{array}$$

$$\begin{array}{r} 1\ 1 \\ 1 \\ + 1 \\ \hline 1 \end{array}$$

Carry into the
next column.

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

$$\begin{array}{r} 10011 \\ - 01110 \\ \hline \end{array}$$

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

$$\begin{array}{r} 1001\color{red}{1} \\ - 0111\color{red}{0} \\ \hline \color{red}{1} \end{array}$$

One minus zero
is one.

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

$$\begin{array}{r} 10011 \\ - 01110 \\ \hline 01 \end{array}$$

One minus one
is zero.

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

$$\begin{array}{r} 10011 \\ - 01110 \\ \hline ?01 \end{array}$$

Zero minus one?
We have to borrow
from the left.
(that's like borrowing 2)

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

Remember to
subtract an extra 1
in this column.

$$\begin{array}{r} 1 \\ -1 1 \\ 10\color{red}{0}11 \\ - 01\color{red}{1}10 \\ \hline \color{red}{1}01 \end{array}$$

0 (plus the 2 we
borrowed)
minus 1
is ... 1.

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

$$\begin{array}{r} \\ 1 \\ 1 1 1 \\ - 0 1 1 0 \\ \hline 1 0 1 \end{array}$$

Zero
Minus one
Minus the one we
borrowed.

... we'll have to
borrow from the
column to the left.

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

$$\begin{array}{r} \\ \\ -1 -1 \\ 1\cancel{0}011 \\ - \cancel{0}1110 \\ \hline 0101 \end{array}$$

Now we have zero
Plus the 2 we borrowed
from the left
Minus 1 (below)
Minus the 1 we borrowed
from the right (above)

That leaves zero.

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

$$\begin{array}{r} 1 \\ 1 \\ -11 \\ \hline 10011 \\ - 01110 \\ \hline 00101 \end{array}$$

Finally, we have one
Minus the one we
borrowed

Subtraction in Binary

- Subtraction is similar, it works right to left
 - Least significant bit to most significant

That leaves 00101
(or, 5 in base 10)

$$\begin{array}{r} \\ 1 \\ -1 1 \\ 10011 \\ - 01110 \\ \hline 00101 \end{array}$$

We had to borrow
twice.

Rules for Binary Subtraction

- Subtract bits, working from least significant to most significant.
- Propagate borrow into subsequent columns.
- Just 8 possible cases:

Borrow from
the next
column.



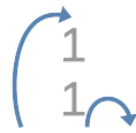
0
- 0
0

0
- 1
1

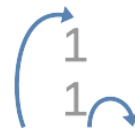
1
- 0
1

1
- 1
0

Borrow from
the previous
column



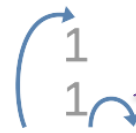
1
- 1
0



1
- 0
1



0
- 1
1



1
- 1
0

Unsigned Type Capacity

- Integer types use a fixed number of bits.
- This limits the range of values they can hold.

00000000

Smallest value an unsigned char can hold.

11111111

Largest value an unsigned char can hold.

00

Smallest value a 32-bit unsigned int can hold.

111

Largest value a 32-bit unsigned int can hold.

Understanding Unsigned Ranges

- So, these unsigned ranges make sense.

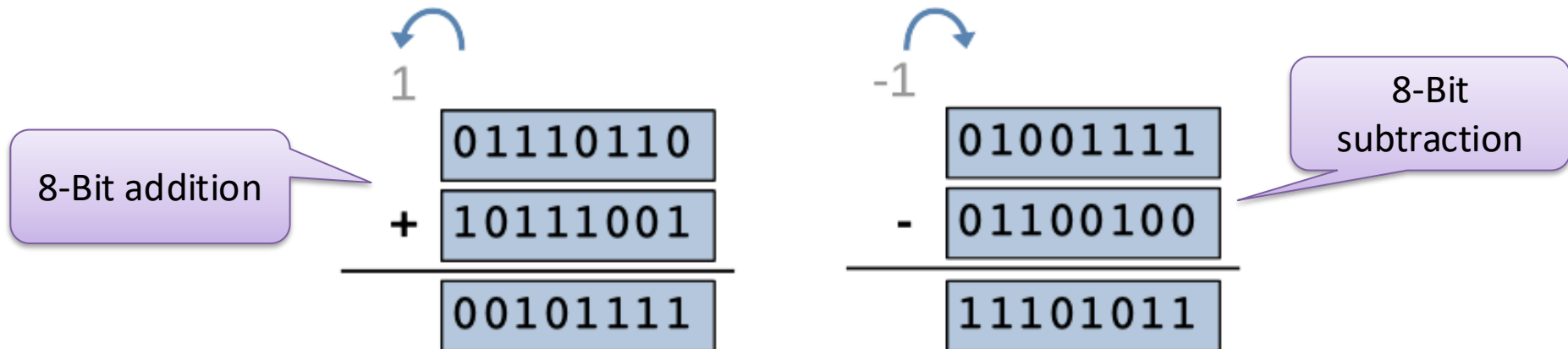
Type	Bits	Minimum	Maximum
signed char	8	-128	127
unsigned char	8	0	255
signed short	16	-32,768	32,767
unsigned short	16	0	65,535
signed int	32	-2,147,483,648	2,147,483,647
unsigned int	32	0	4,294,967,295
signed long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long	64	0	18,446,744,073,709,551,615

And that's 64 bits,

[illegible]

Unsigned Overflow

- Arithmetic operations can exceed capacity.
 - On **addition**, a **carry out of the most significant bit** indicates (unsigned) overflow.
 - On **subtraction**, a **borrow into the most significant bit** indicates (unsigned) overflow.



Overflow During Arithmetic

- In a high-level language, programs don't tell you when an overflow occurs.
- It's the programmer's job to use a type that's large enough.

```
unsigned short x = 0;

while ( true ) {
    printf( "%u\n", x );
    x = x + 10;
}
```

This will overflow after
x is 65530

```
11111111111111010
+ 0000000000001010
-----
10000000000000100
```

Overflow During Type Conversion

- Going from a wider to a narrower type can cause overflow.
- The narrow type may not get all the high-order one bits

```
unsigned int x = 65540;  
unsigned short y;
```

```
y = x;
```

```
printf( "%u\n", y );
```

An unsigned int can hold this, no problem.

This assignment will just get the low-order 16 bits.

00000000000000001000000000000000100 x

These bits are lost



0000000000000000100 y

Signed Value Representation

- The C standard doesn't require a particular way to represent signed values.
- *Two's complement* is what the hardware normally uses to represent signed numbers.

long double 

double 

float 

signed long long 
signed long 
signed int 
signed short 
signed char 

 unsigned long long

 unsigned long

 unsigned int

 unsigned short

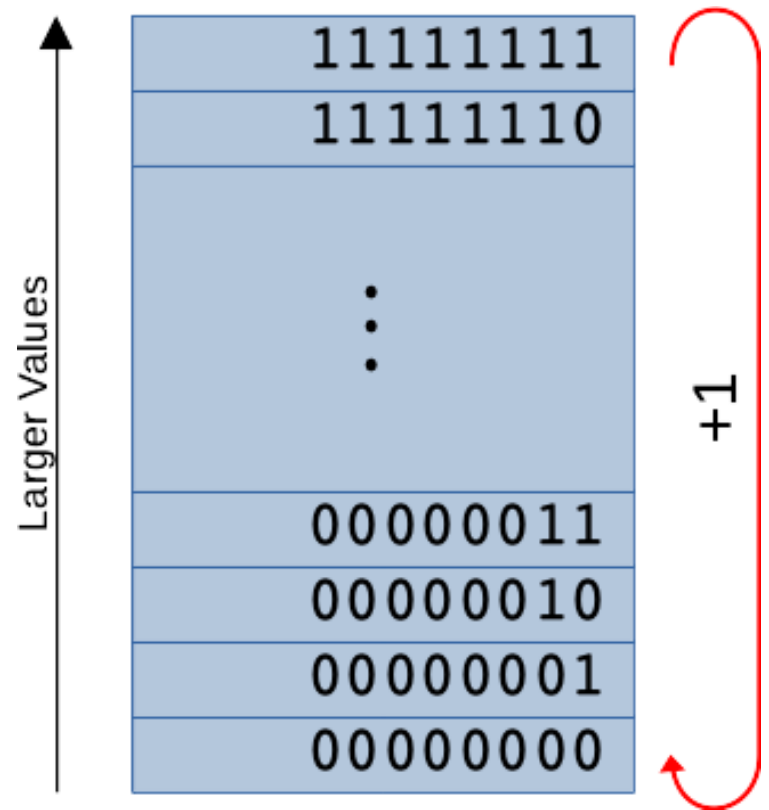
 unsigned char

_Bool 

void •

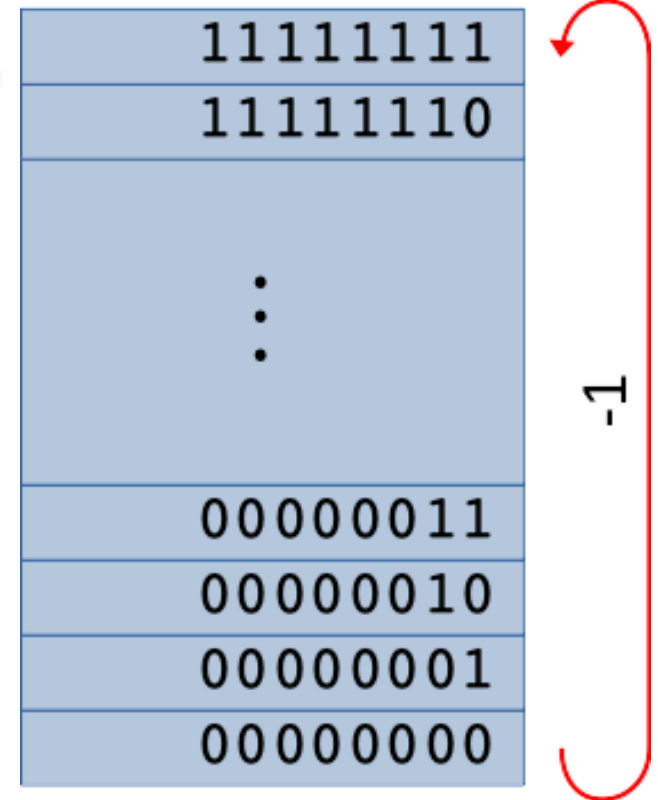
Value Representation

- Unsigned values are represented like this picture.
- Assume you're using a fixed number of bits.
- ... if you add one to the largest value, you end up back at zero.
 - Of course, this is because of overflow.



Value Representation

- Likewise, if you subtract 1 from zero, you get the largest value.
 - Also an example of overflow.
- So, this value works kind of like -1.



Value Representation

- In fact, adding this value is like adding -1.

$$\begin{array}{r} 00000011 \\ + 11111111 \\ \hline 100000010 \end{array}$$

8-bit example.

This bit is lost
because of
overflow.

11111111
11111110
⋮
00000011
00000010
00000001
00000000

+11111111₂

Value Representation

- This also works for other values.
 - This value is kind of like -2.
 - It's what you get for zero minus two.
 - And, added to another value, it gives you a value that's two less.

$$\begin{array}{r}
 00011011 \\
 + 11111110 \\
 \hline
 100011001
 \end{array}$$

Carry out of the high-order bit is lost.



11111111
11111110
⋮
00011011
00011010
00011001
⋮
00000001
00000000

+11111110₂

Signed Value Representation

- This is how *Two's Complement* works

Use these bit patterns

To represent negative values.

11111111
11111110
⋮
10000001
10000000
01111111
⋮
00000001
00000000
11111111
11111110
⋮
10000001
10000000



Signed Value Representation

- All the non-negative values look just like they did with unsigned.
- The leftmost bit tells you if the number is negative (1) or non-negative (0)
- Half the values start with 1
 - That's why we have one extra negative value (compared to positive).

01111111
01111111
⋮
00000001
00000000
11111111
11111110
⋮
10000001
10000000

Integer Ranges on the Common Platform

Type	Bits	Minimum	Maximum
signed char	8	-128	127
unsigned char	8	0	255
signed short	16	-32,768	32,767
unsigned short	16	0	65,535
signed int	32	-2,147,483,648	2,147,483,647
unsigned int	32	0	4,294,967,295
signed long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long	64	0	18,446,744,073,709,551,615
signed long long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807
unsigned long long	64	0	18,446,744,073,709,551,615

One extra negative value.

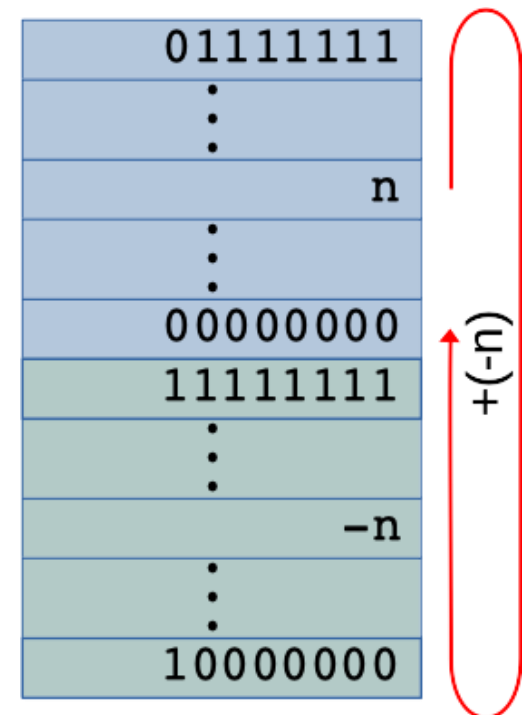
One extra negative value.

One extra negative value.

One extra negative value.

Computing the Two's Complement

- We always want n and $-n$ to add up to zero (with a carry out of the high-order bit)
- That's our definition of two's complement
 - For an 8-bit value:
 - We $-n$ such that:



$$\begin{array}{r} n \\ + \quad -n \\ \hline 1000000000 \end{array}$$

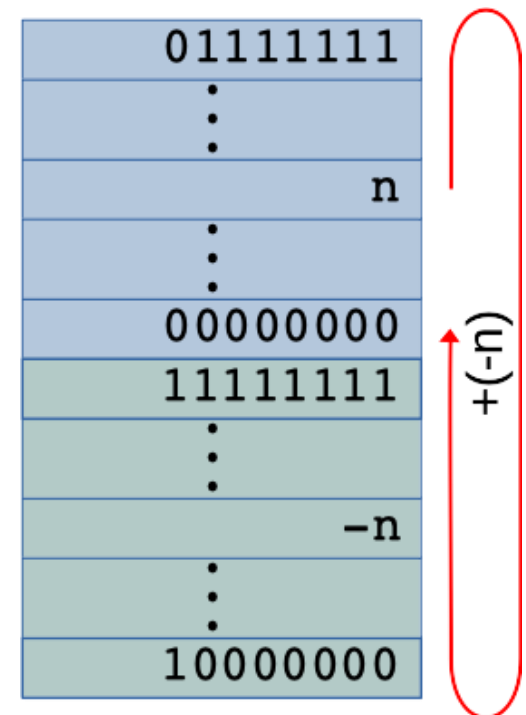
Carry out of the leftmost bit.

Computing the Two's Complement

- We always want n and $-n$ to add up to zero (with a carry out of the high-order bit)
- That's our definition of two's complement
 - For an 8-bit value:
 - We $-n$ such that:

Equivalently

$$\begin{array}{r} 100000000 \\ - \quad \quad n \\ \hline -n \end{array}$$

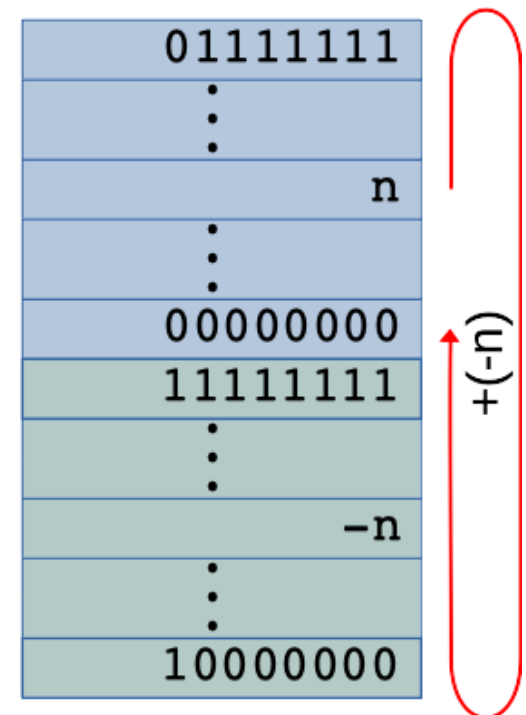


Computing the Two's Complement

- We always want n and $-n$ to add up to zero (with a carry out of the high-order bit)
- That's our definition of two's complement
 - For a **d-bit** value:
 - We $-n$ such that:

$$\begin{array}{r} \text{d zero bits} \\ \overbrace{10000\dots00000} \\ - \\ \hline n \\ -n \end{array}$$

I'm just 2^d



Computing the Two's Complement

- The negation of a value is called its *two's complement*
- Let's try an example
 - Calculate -10 as an 8-bit two's complement value.

$$\begin{array}{r} 100000000 \\ - 00001010 \\ \hline 11110110 \end{array}$$

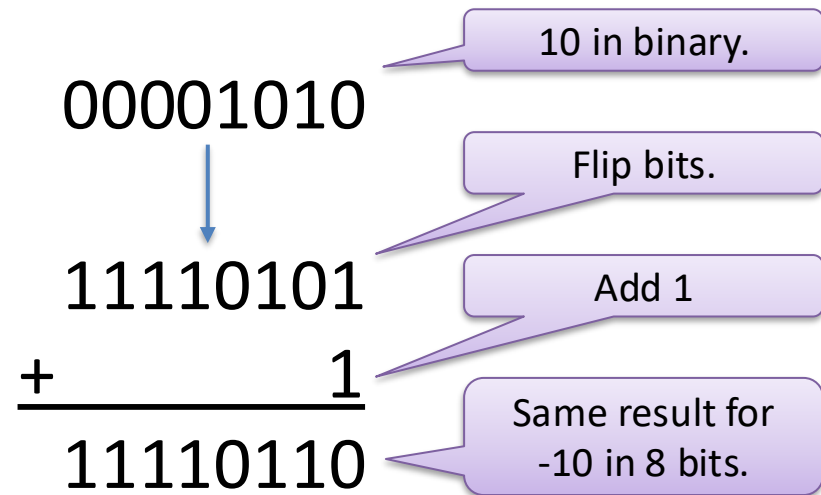
I'm a 1 followed by 8 zeros.

I'm 10 in binary.

So, this is -10
(as an 8-bit value)

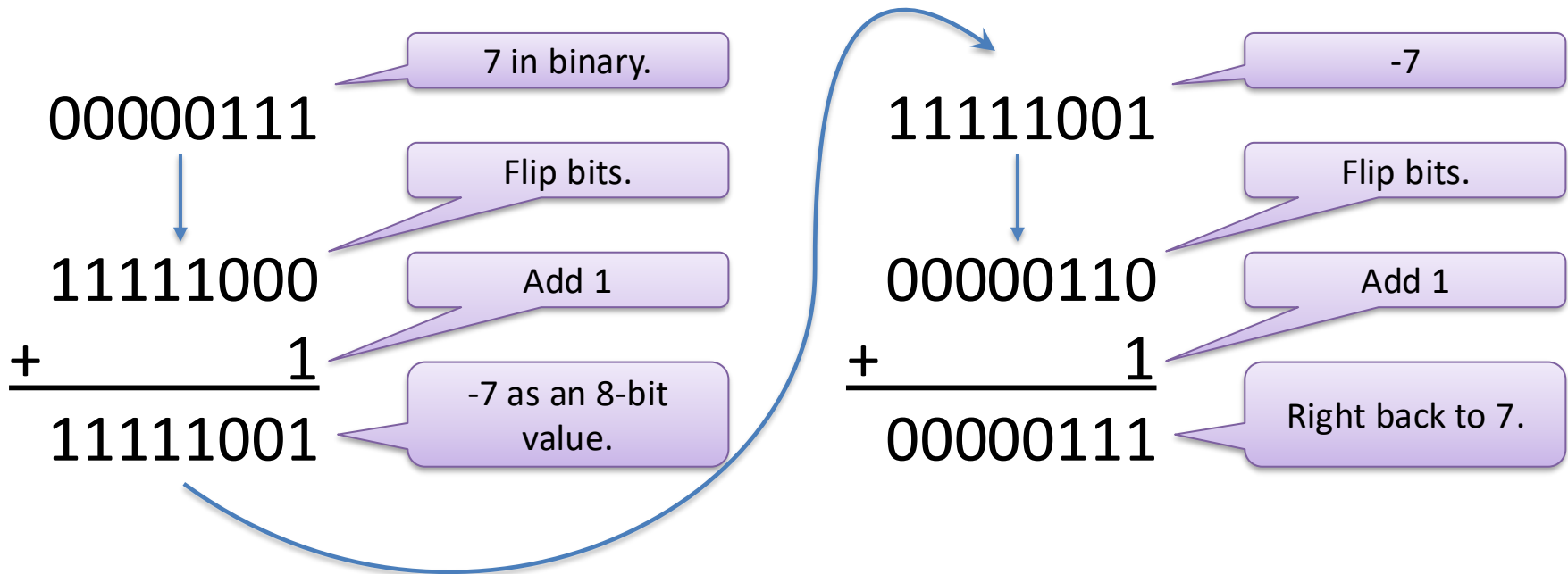
Computing the Two's Complement

- There's another way to compute the two's complement.
- The easy way
 - Flip all bits $0 \leftrightarrow 1$
 - Then add one.
 - Same result, but maybe easier to work out.



It Works Both Ways

- The two's complement operation will negate positive or negative values.
 - Even one that's already negative.



Length Matters

- Notice that the representation for a negative number depends on the number of bits.

$$\begin{array}{r} 00011001 \\ \downarrow \\ 11100110 \\ + \quad \quad 1 \\ \hline 11100111 \end{array}$$

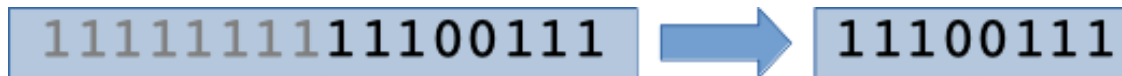
-25 as an 8-bit
value.

$$\begin{array}{r} 00000000000011001 \\ \downarrow \\ 11111111111100110 \\ + \quad \quad \quad 1 \\ \hline 11111111111100111 \end{array}$$

-25 as a 16-bit
value.

Sign Extension

- The compiler has to take this into account when converting signed types:
- Wider signed integer to a narrower one:
Just take the low-order bits.



- Narrower signed integer to a wider one:
Fill high-order bits with a copy of the leftmost bit.



- This is called *sign extension*

Signed Arithmetic

- With two's complement, we can add and subtract just like we did with unsigned values.
 - But, a carry out of the leftmost bit doesn't necessarily indicate overflow.

$$\begin{array}{r} 00010100 \\ + 11110101 \\ \hline 00001001 \end{array}$$

20

-11

9

Carry out of the leftmost bit.
(but no overflow)

$$\begin{array}{r} 11101010 \\ - 11110111 \\ \hline 11110011 \end{array}$$

-22

-9

-13

Borrow into the leftmost bit
(but no overflow)

Signed Overflow

- How can we detect signed overflow?
- These examples couldn't possibly overflow.
 - A positive plus negative.
 - A negative minus a negative.

$$\begin{array}{r} 00010100 \\ + 11110101 \\ \hline 00001001 \end{array}$$

20

-11

9

$$\begin{array}{r} 11101010 \\ - 11110111 \\ \hline 11110011 \end{array}$$

-22

-9

-13

Signed Overflow

- These could overflow
 - A negative plus a negative
 - A negative minus a positive
- You can tell from the sign that the result is wrong.

$$\begin{array}{r} 10101100 \\ + 11001101 \\ \hline 01111001 \end{array}$$

-84

-51

Oops
That's 121

$$\begin{array}{r} 10111000 \\ - 01000100 \\ \hline 01110100 \end{array}$$

-72

68

Oops
That's 116

Signed Overflow

- Or these
 - A positive plus a positive
 - A positive minus negative
- You can tell from the sign that the result is wrong.

$$\begin{array}{r} 01100011 \\ + 00100011 \\ \hline 10000110 \end{array}$$

99

35

Oops
That's -122

$$\begin{array}{r} 01010100 \\ - 10011001 \\ \hline 10111011 \end{array}$$

84

-103

Oops
That's -69

Signed Overflow

- You can also detect overflow by watching the carry / borrow behavior.
 - For add, the carry into the sign bit should always be the same as the carry out.
 - For subtract, the borrow from the sign bit should always be the same as the borrow into it.
 - Otherwise, you have an overflow.

Diagram illustrating signed addition with overflow. The numbers are represented in 8-bit two's complement:

$$\begin{array}{r} 10101100 \\ + 11001101 \\ \hline 01111001 \end{array}$$

A blue arrow indicates a carry into the sign bit (bit 7). The result is 01111001.

Annotations:

- 84 (for 10101100)
- 51 (for 11001101)
- Oops That's 121 (for the result 01111001)

Diagram illustrating signed subtraction with overflow. The numbers are represented in 8-bit two's complement:

$$\begin{array}{r} 10111000 \\ - 01000100 \\ \hline 01110100 \end{array}$$

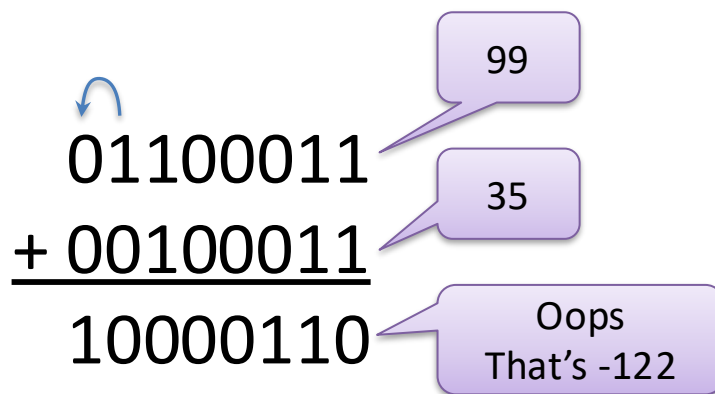
A blue arrow indicates a borrow into the sign bit (bit 7). The result is 01110100.

Annotations:

- 72 (for 10111000)
- 68 (for 01000100)
- Oops That's 116 (for the result 01110100)

Signed Overflow

- You can also detect overflow by watching the carry / borrow behavior.
 - For add, the carry into the sign bit should always be the same as the carry out.
 - For subtract, the borrow from the sign bit should always be the same as the borrow into it.
 - Otherwise, you have an overflow.



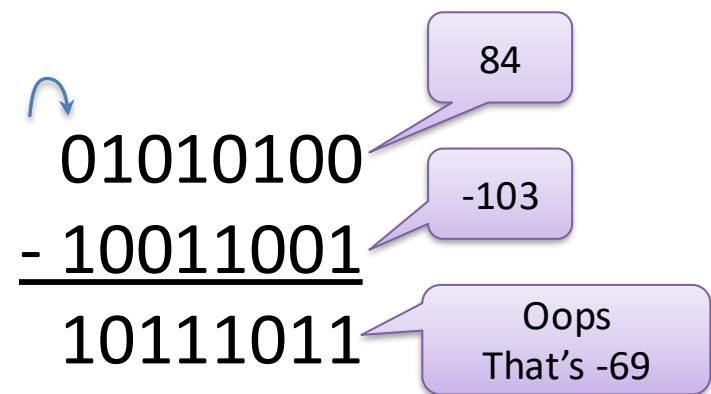
A binary addition diagram. The first number is 01100011, with a callout bubble indicating its decimal value is 99. The second number is 00100011, with a callout bubble indicating its decimal value is 35. A blue curved arrow points from the carry-out of the sign bit (the leftmost bit) to the carry-in of the sign bit, indicating they are different. The result is 10000110, with a callout bubble stating "Oops That's -122".

$$\begin{array}{r} 01100011 \\ + 00100011 \\ \hline 10000110 \end{array}$$

99

35

Oops
That's -122



A binary subtraction diagram. The first number is 01010100, with a callout bubble indicating its decimal value is 84. The second number is 10011001, with a callout bubble indicating its decimal value is -103. A blue curved arrow points from the borrow-out of the sign bit (the leftmost bit) to the borrow-in of the sign bit, indicating they are different. The result is 10111011, with a callout bubble stating "Oops That's -69".

$$\begin{array}{r} 01010100 \\ - 10011001 \\ \hline 10111011 \end{array}$$

84

-103

Oops
That's -69

Signed Overflow

- When there's no overflow the carry / borrow into / out of the leftmost bit match.
- Here are our two examples from earlier.

Carry in and
carry out.

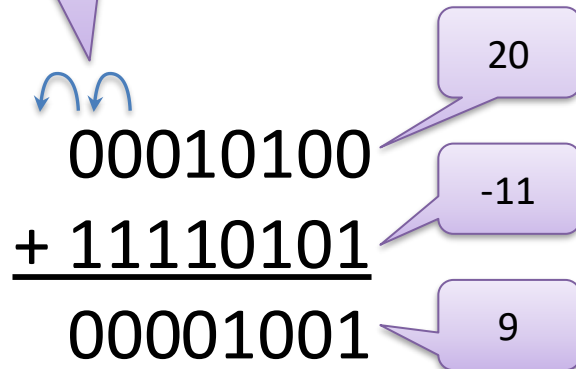


Diagram illustrating binary addition with a carry in and carry out. The numbers are 00010100 (20) and 11110101 (-11). The result is 00001001 (9). Blue arrows indicate the carry in from the left and the carry out to the left.

$$\begin{array}{r} 00010100 \\ + 11110101 \\ \hline 00001001 \end{array}$$

20
-11
9

Borrow out and
borrow in.

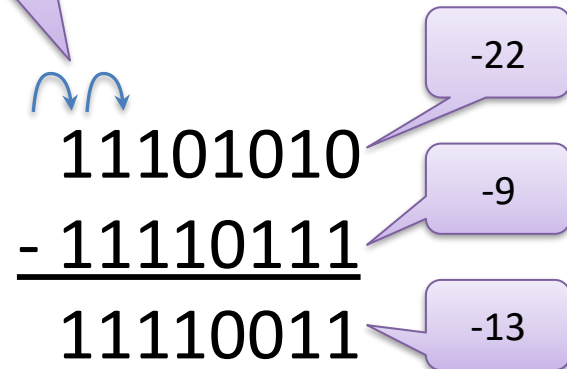


Diagram illustrating binary subtraction with a borrow out and borrow in. The numbers are 11101010 (-22) and 11110111 (-9). The result is 11110011 (-13). Blue arrows indicate the borrow in from the left and the borrow out to the left.

$$\begin{array}{r} 11101010 \\ - 11110111 \\ \hline 11110011 \end{array}$$

-22
-9
-13

Signed Overflow

- When there's no overflow the carry / borrow into / out of the leftmost bit match.
- Here are our **two more examples**.

No carry in and
carry out.

$$\begin{array}{r} 00011100 \\ + 00100101 \\ \hline 01000001 \end{array}$$

28

37

65

No borrow out
and borrow in.

$$\begin{array}{r} 11010010 \\ - 00010001 \\ \hline 11000101 \end{array}$$

-42

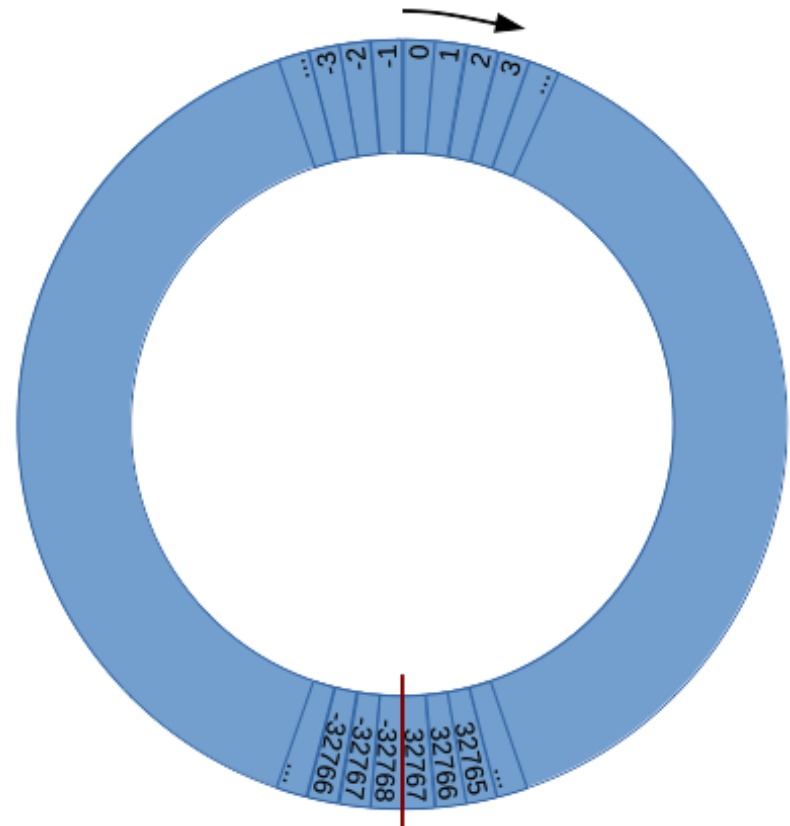
17

-59

Overflow

- Like most languages, C doesn't detect overflow.
- It's the programmer's job to think about this.

```
// So, what will happen?  
short x = 0;  
while ( true ) {  
    x = x + 1;  
    printf( "%d\n", x );  
}
```



Overflow Happens

- Overflow can happen when you ...
 - Compute a value that's outside the range of the type that's holding it.

```
int x = 65536;  
x = x * x;
```

Ouch

- Copy from a wider type to a narrower one.

```
int x = 123456789;  
short y = x;
```

Ouch

Overflow Happens

- Overflow can happen when you ...
 - Copy between signed and unsigned types of the same width.
 - From signed to unsigned:

```
short x = -1;  
unsigned short y = x;
```

1111111111111111
looks different as an
unsigned vs a signed
value.

- Or, the other way around:

```
unsigned short x = 40000;  
short y = x;
```

This is
1001110001000000
It will look like a
negative value.

Overflow Happens

- Even just changing the sign of a value can cause overflow ... in one, special case.

```
int x = -2147483648;  
x = -x;
```

I just barely fit in
an int.

But not me.

Writing Down Binary Numbers

- A big binary number can be hard to remember
 - ... and take a lot of space to write down.
 - Hexadecimal is a compact way of writing binary numbers.
 - Each digit represents four bits.

Binary: 110100101010

 ↓ ↓ ↓

 1101 0010 1010

 ↓ ↓ ↓

Hexadecimal: D 2 A

A 12-bit number

Only 3 digits in hexadecimal

Binary Numbers in Hexadecimal

- Hexadecimal needs 16 possible digit values.
- One for each possible pattern of four bits.

0000 → 0	0001 → 1	0010 → 2	0011 → 3
0100 → 4	0101 → 5	0110 → 6	0111 → 7
1000 → 8	1001 → 9	1010 → A	1011 → B
1100 → C	1101 → D	1110 → E	1111 → F

Binary:

1101011111

A 10-bit number

0011 0101 1111

Hexadecimal:

3

5

F

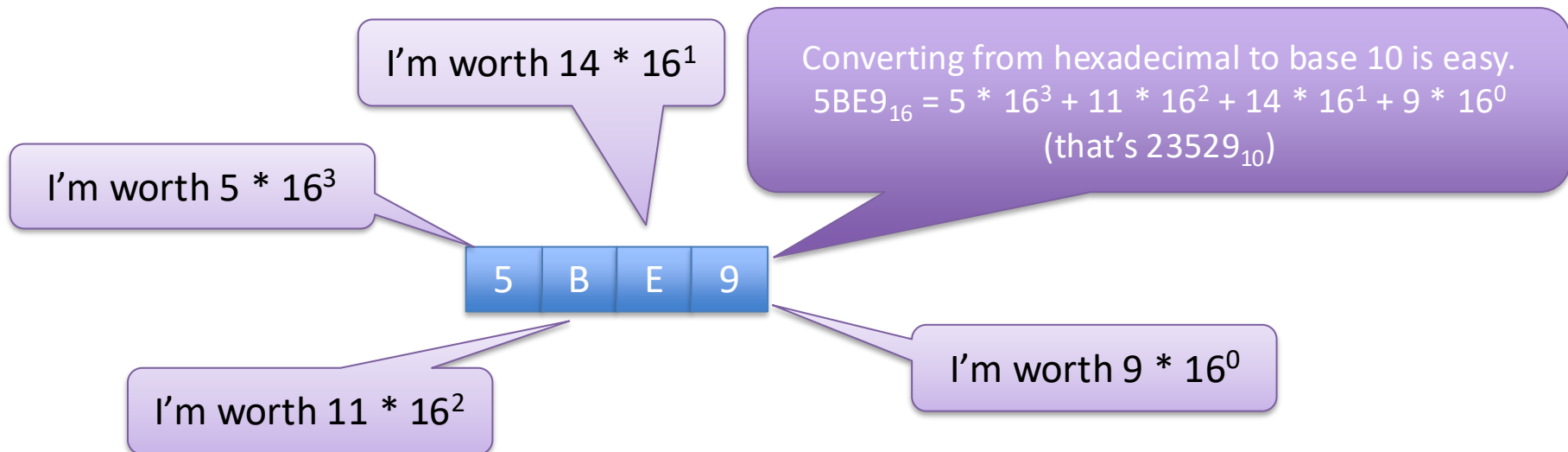
Still 3 digits in hexadecimal

Hexadecimal Numbers as Base 16

- Hexadecimal is a positional number system

- Using base 16
- With 16 digit values
- Each digit is worth 16 times the digit to the right
- We use notation like $5BE9_{16}$ to show a number is hexadecimal

0 is worth 0	1 is worth 1	2 is worth 2	3 is worth 3
4 is worth 4	5 is worth 5	6 is worth 6	7 is worth 7
8 is worth 8	9 is worth 9	A is worth 10	B is worth 11
C is worth 12	D is worth 13	E is worth 14	F is worth 15



Decimal to Hexadecimal

- To convert to hexadecimal, we can use a technique like we used for decimal $\rightarrow$ binary.

```
Given some value v
```

```
While v is non-zero
```

```
    Next hex digit is  $v \% 16$ 
```

```
    Let  $v = v / 16$  to  
        discard rightmost hex digit  
        and shift remaining digits right
```

Math in Hexadecimal

- You can do arithmetic with numbers in hexadecimal.
- How? There are some alternatives.
 - Convert to binary
 - Consider the base-10 value of each digit
 - Perform the arithmetic directly in base 16

$$\begin{array}{r} 3A \\ + \underline{C9} \end{array}$$

Math in Hexadecimal

- Alternatives

- **Convert to binary**

Back-and-forth to hexadecimal is easy.

- Consider the base-10 value of each digit

- Perform the arithmetic directly in base 16

$$\begin{array}{r} 3A \\ + B9 \\ \hline \end{array} \quad \longrightarrow \quad \begin{array}{r} 00111010 \\ + 10111001 \\ \hline 11110011 \end{array} \quad \longrightarrow \quad \begin{array}{r} 3A \\ + B9 \\ \hline F3 \end{array}$$

Math in Hexadecimal

- Alternatives

- Convert to binary
- **Consider the base-10 value of each digit**
- Perform the arithmetic directly in base 16

You already know how to do math on small numbers in base 10

3A
+ B9

That's 10 + 9 in decimal.
That's 19.

Math in Hexadecimal

- Alternatives
 - Convert to binary
 - **Consider the base-10 value of each digit**
 - Perform the arithmetic directly in base 16

$$\begin{array}{r} 1 \\ 3A \\ + B9 \\ \hline 3 \end{array}$$

Carry into the next column (16), leaving 3 in this column.

Math in Hexadecimal

- Alternatives
 - Convert to binary
 - **Consider the base-10 value of each digit**
 - Perform the arithmetic directly in base 16

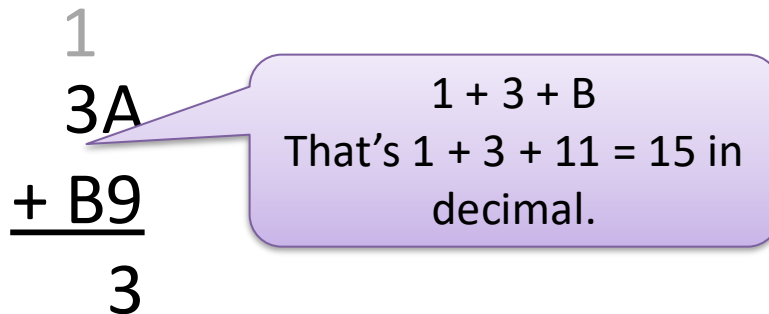


Diagram illustrating the addition of hexadecimal numbers 13A and B9, resulting in 3. The calculation is shown as:

$$\begin{array}{r} 1 \\ 3A \\ + B9 \\ \hline 3 \end{array}$$

A callout box explains the base-10 conversion of the digits:

1 + 3 + B
That's 1 + 3 + 11 = 15 in decimal.

Math in Hexadecimal

- Alternatives
 - Convert to binary
 - **Consider the base-10 value of each digit**
 - Perform the arithmetic directly in base 16

$$\begin{array}{r} 1 \\ 3A \\ + B9 \\ \hline F3 \end{array}$$

We write 15 as F in hexadecimal.

Math in Hexadecimal

- Alternatives

- Convert to binary
- Consider the base-10 value of each digit
- **Perform the arithmetic directly in base 16**

You could learn single-digit sums for hexadecimal

$$\begin{array}{r} 3A \\ + B9 \\ \hline \end{array}$$

A table might help.

Math in Hexadecimal

- Alternatives
 - Convert to binary
 - Consider the base-10 value of each digit
 - **Perform the arithmetic directly in base 16**

$$\begin{array}{r} 1 \\ 3A \\ + B9 \\ \hline 3 \end{array}$$

A + 9 is 3
With a carry into the
next column.

Math in Hexadecimal

- Alternatives
 - Convert to binary
 - Consider the base-10 value of each digit
 - **Perform the arithmetic directly in base 16**

$$\begin{array}{r} 1 \\ 3A \\ + B9 \\ \hline F3 \end{array}$$

3 + B plus a carry
equals F

Two-Digit Sums in Hexadecimal

+	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10
2	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11
3	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12
4	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13
5	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14
6	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15
7	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16
8	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17
9	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18
A	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19
B	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A
C	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B
D	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
E	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
F	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E

Red spaces have a carry into the next column.

Two-Digit Sums in Hexadecimal With Carry

c+	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10
1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11
2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12
3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13
4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14
5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15
6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16
7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17
8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18
9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19
A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A
B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B
C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E
F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E	1F

This table assumes a carry in from the right.

Binary Numbers as Octal

- Octal is another shorthand way to write binary
 - With each digit representing 3 bits
 - Less compact .. but you don't need to use letters.

000 → 0	001 → 1
010 → 2	011 → 3
100 → 4	101 → 5
110 → 6	111 → 7

Binary: 110100101010

 ↓ ↓ ↓ ↓

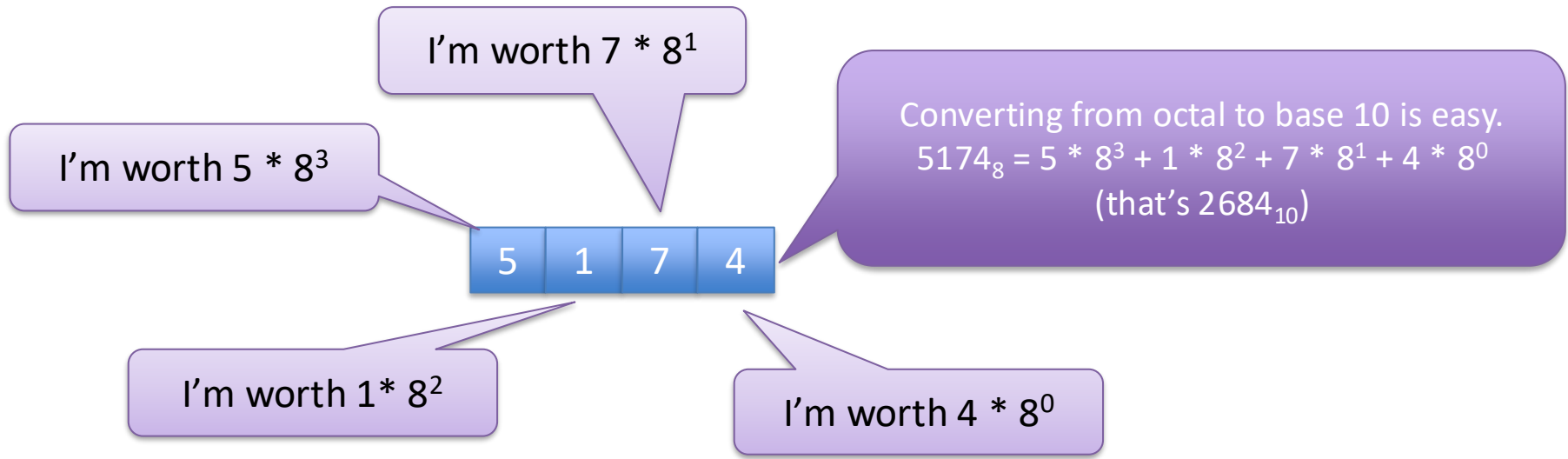
 110 100 101 010

 ↓ ↓ ↓ ↓

Octal: 6 4 5 2

Octal Numbers as Base 8

- Octal is a base-8 representation
 - 8 possible digit values
 - Each digit worth 8 times the digit to its right.
 - We use notation 5174_8 to show that a number is octal



Octal and Hexadecimal in C

- In C, you can give an octal constant by starting it with a 0 (zero).

```
int a = 0123;
```

- Or, a hexadecimal constant by starting it with 0x (zero-x)

```
int b = 0xABC;
```

Either capital or
lower-case is OK.

- Java also lets us do this, but maybe you never used it.

More Conversion Specifiers

- `printf()` and `scanf()` can read and write values in these formats.

Conversion Specifier	Input / Output
<code>%o</code>	An integer in octal
<code>%x</code>	An integer in hexadecimal
<code>%X</code>	An integer in hexadecimal (capitalized)
<code>%u</code>	An unsigned integer
<code>%lli</code>	A long long int (decimal or any of the formats above)